



Informe Final de Trabajo Profesional de Ingeniería en Informática

Oxidized Neural Orchestra

Sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce.

Tutor: Ing. Ricardo Andrés Veiga
rveiga@fi.uba.ar

Co-Tutor y Orientador: Dr. Ing. José Ignacio Alvarez Hamelin
ihameli@fi.uba.ar

Alumnos

Alejo Ordoñez (*Padrón 108397*)
alordonez@fi.uba.ar

Lorenzo Minervino (*Padrón 107863*)
lminervino@fi.uba.ar

Marcos Bianchi Fernández (*Padrón 108921*)
mbianchif@fi.uba.ar

Facultad de Ingeniería, Universidad de Buenos Aires

Índice

Aval del director	5
Agradecimientos	6
1 Resumen	7
2 Palabras clave	8
3 Abstract	9
4 Keywords	10
5 Introducción	11
6 Estado del arte	12
7 Problema detectado	14
8 Solución implementada	15
8.1 Visión general	15
8.2 Capa de comunicación	16
8.2.1 Protocolo de dos planos	16
8.2.2 Transporte y reducción del tráfico	16
8.2.3 Handles tipados	17
8.3 Motor de redes neuronales	17
8.3.1 El modelo como buffer plano	17
8.3.2 Componentes	18
8.4 Los tres algoritmos	19
8.4.1 Parameter Server	19
8.4.2 Ring All-Reduce	19
8.4.3 Strategy Switch	20
8.5 Plano de control	20
8.5.1 Selección de topología	20
8.5.2 Sesión orientada a eventos	21
8.6 Interfaces	21
8.7 Estado de la implementación	21
9 Metodología aplicada	23
9.1 Gestión y roles	23
9.2 Proceso de desarrollo	23
9.3 Seguimiento, tickets y gestión del alcance	23
9.4 Automatización	24
9.5 Criterios de aceptación	24
9.6 Artefactos e indicadores	24

10 Herramientas externas utilizadas	26
10.1 Lenguajes y entornos	26
10.2 Bibliotecas de Rust	26
10.3 Bibliotecas de Python	27
10.4 Documentación y edición del informe	28
10.5 Herramientas de inteligencia artificial generativa	28
10.6 Consideración sobre licencias	29
11 Experimentación y validación	30
11.1 Pruebas automatizadas	30
11.2 Entorno de experimentación y reproducibilidad	30
11.3 Estudio I: Parameter Server frente a All-Reduce	31
11.3.1 Pregunta e hipótesis	31
11.3.2 Fundamento	31
11.3.3 Criterio de comparación justa	32
11.3.4 Resultados: convergencia	33
11.3.5 Resultados: throughput y escalabilidad	35
11.3.6 Resultados: presión de comunicación	35
11.3.7 Discusión y matriz de decisión	37
11.3.8 Limitaciones y conclusión del estudio	38
11.4 Estudio II: impacto de las épocas offline	38
11.4.1 Pregunta e hipótesis	38
11.4.2 Metodología	38
11.4.3 Resultados: tiempo de ejecución	40
11.4.4 Resultados: exactitud	40
11.4.5 Resultados: inestabilidad de la pérdida	42
11.4.6 Discusión y limitaciones	43
11.4.7 Conclusión del estudio	43
11.5 Estudio III: Strategy Switch	44
11.6 Síntesis de la validación	44
12 Cronograma de las actividades realizadas	45
12.1 Fases efectivamente recorridas	45
12.2 Contraste entre lo planificado y lo ejecutado	45
12.3 Carga horaria efectiva	46
13 Riesgos materializados y lecciones aprendidas	48
13.1 Riesgos previstos que se materializaron	48
13.2 Riesgos no previstos	48
13.3 Desvíos de alcance	49
13.4 Lecciones aprendidas	50
14 Impactos económico, social y ambiental del proyecto	52
15 Trabajos futuros	54

16 Conclusiones	56
17 Aportes individuales	57
18 Referencias	59
19 Anexos	61
19.1 Anexo A: Repositorio del proyecto	61
19.2 Anexo B: Documentación técnica	61
19.3 Anexo C: Reproducibilidad de los experimentos	62
19.4 Anexo D: Monografías	62

Aval del director

En Buenos Aires a [día] de [mes] del año [año] se reúnen el Ing. Ricardo Andrés Veiga con los estudiantes de la carrera de Ingeniería en Informática: el Sr. Alejo Ordoñez (DNI 44480134), el Sr. Lorenzo Minervino (DNI 42720539) y el Sr. Marcos Bianchi Fernández (DNI 43874791). El objetivo de la reunión es revisar el Informe Final del Trabajo Profesional de Ingeniería en Informática.

Luego de haber leído el informe del Trabajo Profesional “Oxidized Neural Orchestra: sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce”, el Director del mismo, Ing. Ricardo Andrés Veiga, declara que el mismo se corresponde con el trabajo realizado a lo largo del proyecto y avala el documento presentado como Informe Final, tanto en su completitud como en la claridad de redacción.

Firma y aclaración del director:

Ing. Ricardo Andrés Veiga _____

Firmas y aclaraciones de los estudiantes:

Alejo Ordoñez _____

Lorenzo Minervino _____

Marcos Bianchi Fernández _____

Agradecimientos

A nuestro tutor, el Ing. Ricardo A. Veiga, por acompañar el trabajo de punta a punta y por sostener la exigencia sobre la forma del documento y sobre el rigor de lo que se afirma en él.

A nuestro co-tutor y orientador, el Dr. Ing. José Ignacio Alvarez-Hamelin, y al CoNexDat, Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería, por el marco en el que se desarrolló la Práctica Profesional Supervisada y por orientar las decisiones de diseño del sistema distribuido.

A nuestras familias y amigos, que bancaron los dos cuatrimestres que llevó este trabajo.

1 Resumen

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución; los algoritmos que se utilizan hacen uso intensivo de CPU y GPU, y requieren de grandes capacidades de memoria para almacenar tanto los datos de entrenamiento como el modelo en sí. Junto con la revolución de la aplicación de la inteligencia artificial, minimizar ese tiempo de entrenamiento se ha vuelto un tema central, y la estrategia más escalable para lograrlo es su ejecución distribuida.

Distribuir el entrenamiento, sin embargo, no es directo: la estrategia de distribución que se elija condiciona tanto el tiempo total de ejecución como la convergencia de la optimización. En este trabajo se implementó *Oxidized Neural Orchestra* (O.N.O.), un sistema distribuido de entrenamiento en Rust que incluye una biblioteca de redes neuronales propia, construida directamente sobre la biblioteca de arreglos numéricos *ndarray* y sin recurrir a ningún framework de aprendizaje profundo existente. Sobre esa base se implementaron los tres algoritmos que sirven de referencia en el área, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, junto con tres interfaces de uso: una interfaz interactiva de terminal, una biblioteca de Python construida sobre una interfaz de funciones externas, y un binario de ejecución desatendida.

El sistema resultante es parametrizable y permitió comparar las estrategias entre sí bajo criterios de comparación controlados. Los estudios experimentales realizados sobre MNIST y FashionMNIST muestran que, en un entorno homogéneo y sincrónico simulado sobre una única máquina, *All-Reduce* y *Parameter Server* convergen a la misma exactitud, mientras que *All-Reduce* la alcanza en menos tiempo porque el servidor central introduce un punto de serialización; y que postergar la sincronización entre nodos mediante épocas offline degrada la exactitud final de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva.

2 Palabras clave

Aprendizaje profundo, entrenamiento distribuido, sistemas distribuidos, paralelismo de datos, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, convergencia, escalabilidad, Rust.

3 Abstract

Training deep learning models requires large computational resources and long execution times; the algorithms involved make intensive use of the CPU and demand very large memory capacities in order to store both the training data and the model itself. Together with the revolution in the application of artificial intelligence, minimizing this training time has become a pressing topic. To achieve this, the most scalable strategy is distributed execution.

Distributing the training, however, is not straightforward: the distribution strategy that is chosen conditions both the total execution time and the convergence of the optimization. In this work we implemented *Oxidized Neural Orchestra* (O.N.O.), a distributed training system written in Rust that includes a neural network library of our own, built directly on top of the numerical array library *ndarray* and without relying on any existing deep learning framework. On that foundation we implemented the three algorithms that serve as a reference in the area, *Parameter Server*, ring *All-Reduce* and *Strategy-Switch*, together with three user interfaces: an interactive terminal user interface, a Python library built on a foreign function interface, and a headless binary.

The resulting system is parameterizable and made it possible to compare the strategies against each other under controlled fairness criteria. The experimental studies carried out on MNIST and FashionMNIST show that, in a homogeneous and synchronous environment simulated on a single machine, *All-Reduce* and *Parameter Server* converge to the same accuracy, while *All-Reduce* reaches it in less time because the central server introduces a serialization bottleneck; and that delaying synchronization between nodes through offline epochs degrades final accuracy monotonically without providing, in that environment, the time gain that motivates it.

4 Keywords

Deep learning, distributed training, distributed systems, data parallelism, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, convergence, scalability, Rust.

5 Introducción

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución, y la estrategia más escalable para reducirlos es su ejecución distribuida. Sin embargo, distribuir el entrenamiento no es directo: introduce desafíos en el diseño de la estrategia de distribución y puede degradar la convergencia de la optimización si no se realiza con cuidado. Este trabajo se propuso estudiar en profundidad las principales estrategias de entrenamiento distribuido y, a partir de dicho análisis, implementar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo, incluida su propia biblioteca de redes neuronales, que sirviera como base para la comparación de esas estrategias y como punto de partida para el desarrollo de posibles mejoras.

El resultado es *Oxidized Neural Orchestra* (en adelante O.N.O.), un sistema distribuido escrito íntegramente en Rust que implementa tres algoritmos de entrenamiento distribuido, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, sobre una infraestructura de comunicación y un motor de redes neuronales propios. Sobre ese sistema se realizaron los estudios experimentales que constituyen la validación del trabajo.

Conviene señalar desde el comienzo una particularidad de este informe. A diferencia de un trabajo cuyo objetivo es resolver un problema de negocio, aquí el producto es un *sistema de experimentación*: el valor no está solo en que el sistema funcione, sino en la evidencia que permite producir. Por eso la sección de experimentación ocupa un lugar central y está organizada como tres estudios comparativos independientes, cada uno con su propia pregunta de investigación, su metodología y sus conclusiones acotadas.

El presente documento se organiza de la siguiente manera. En el *Estado del arte* se relevan los avances relacionados al entrenamiento distribuido y se presentan los enfoques fundamentales que sirven de base al trabajo. En *Problema detectado* se precisa la dificultad central que se aborda. En *Solución implementada* se describe en detalle el sistema construido, su arquitectura y las decisiones de diseño que la explican. Luego, en *Metodología aplicada* se detalla el proceso de desarrollo efectivamente seguido, y en *Herramientas externas utilizadas* se declaran las tecnologías empleadas y el criterio con que fueron elegidas. En *Experimentación y validación* se presentan los tres estudios comparativos realizados sobre el sistema. Siguen el *Cronograma de las actividades realizadas*, los *Riesgos materializados y lecciones aprendidas*, los *Impactos económico, social y ambiental*, los *Trabajos futuros* y las *Conclusiones*. Finalmente se incluyen los *Aportes individuales*, las *Referencias* y los *Anexos*.

6 Estado del arte

El entrenamiento distribuido surge como una extensión de las técnicas de paralelización en *HPC* (High Performance Computing). Desde los primeros enfoques de paralelismo de datos en frameworks como TensorFlow (TensorFlow Developers, 2021) y PyTorch (Paszke et al., 2019) hasta sus implementaciones más recientes optimizadas para arquitecturas heterogéneas, la evolución de estos métodos refleja la tensión constante entre escalabilidad y coherencia de los parámetros del modelo. Los relevamientos de Ben-Nun y Hoefer (Ben-Nun & Hoefer, 2019) y de Dehghani y Yazdanparast (Dehghani & Yazdanparast, 2023) sistematizan esa tensión y sirvieron de mapa conceptual para este trabajo.

El equilibrio entre convergencia y eficiencia de comunicación ha motivado una amplia línea de investigación. En la actualidad, existen dos enfoques fundamentales que sirven como base para el desarrollo de nuevas técnicas: Parameter Server (M. Li et al., 2014) y All-Reduce (Z. Li, Davis, & Jarvis, 2017).

En el enfoque de Parameter Server, el modelo se divide en un conjunto de parámetros centralizados (no necesariamente en un único nodo), a los cuales los nodos trabajadores envían sus gradientes y desde los cuales reciben las actualizaciones, que surgen de la agregación de estas contribuciones. Este esquema admite tanto una coordinación sincrónica, en la que el servidor espera los gradientes de todos los trabajadores antes de actualizar el modelo, como una asincrónica, en la que aplica cada gradiente apenas llega; esta última mejora la escalabilidad y reduce el tiempo de entrenamiento, aunque a costa de la coherencia entre copias del modelo. El caso extremo de esa relajación es *Hogwild!* (Niu, Recht, Ré, & Wright, 2011), que directamente abraza las condiciones de carrera sobre los parámetros compartidos y demuestra que, bajo hipótesis de esparsidad, la convergencia se preserva.

En contraste, en All-Reduce todos los nodos intercambian gradientes de manera directa y avanzan de forma sincrónica: en cada paso todos aplican la misma actualización, lo que preserva la coherencia entre réplicas. El precio es el costo de comunicación, que crece con el número de nodos, aunque topologías como el anillo lo atenúan. La formulación en anillo de Patarasuk y Yuan (Patarasuk & Yuan, 2009) es óptima en ancho de banda, ya que el volumen que comunica cada nodo resulta esencialmente independiente de la cantidad de participantes, y es la que popularizó Horovod (Sergeev & Del Balso, 2018) en el ámbito del aprendizaje profundo. Trabajos posteriores como BytePS (Jiang et al., 2020) muestran que, en clústeres heterogéneos, la elección entre ambas familias deja de ser evidente.

Uno de los claros casos de combinación de estos algoritmos es Strategy-Switch (Provatas, Chalas, Konstantinou, & Koziris, 2025), que inicia el entrenamiento de forma sincrónica sobre All-Reduce y, guiado por una regla empírica, continúa con Parameter Server asincrónico una vez que el modelo en entrenamiento se estabiliza; logrando así combinar la precisión del régimen sincrónico de *All-Reduce* con la reducción de tiempo del régimen asincrónico de *Parameter Server*. Este algoritmo es el más reciente de los tres y fue el que motivó originalmente la elección del tema.

Una línea transversal a los tres enfoques es la reducción del volumen de comunicación. Aji y Heafield (Aji & Heafield, 2017) proponen transmitir únicamente los componentes de mayor magnitud del gradiente y acumular localmente el residuo, y Lin et al. (Lin, Han, Mao, Wang, & Dally, 2018) extienden la idea con compensaciones que preservan la convergencia a tasas de compresión muy altas. Una alternativa complementaria es reducir la precisión numérica de los mensajes sin alterar la

precisión del cómputo. Ambas técnicas fueron implementadas en este trabajo.

Finalmente, en el terreno del aprendizaje federado, McMahan et al. (McMahan, Moore, Ramage, Hampson, & Arcas, 2017) formalizan la idea de dejar que cada nodo entrene de forma autónoma durante varias épocas antes de sincronizar, y caracterizan el fenómeno de divergencia entre réplicas que esto induce, conocido como *client drift*. Ese mecanismo, que en O.N.O. se expone bajo el nombre de *épocas offline*, es el objeto de uno de los estudios experimentales de este informe.

7 Problema detectado

El principal desafío del desarrollo de algoritmos distribuidos de entrenamiento de modelos de aprendizaje profundo es encontrar una estrategia que disminuya satisfactoriamente el tiempo del entrenamiento, o que provea una solución a la incapacidad de un flujo existente de almacenar el dataset o incluso el modelo en sí. Esto sería mucho más sencillo si las iteraciones no fueran dependientes; podríamos simplemente utilizar una estrategia análoga a un *fork-join* y combinar la solución. Pero en un *gradient-descent*, la dirección de un descenso en la superficie de costo depende del punto en el que se encuentra el proceso en un momento dado, es decir, depende de la configuración de la matriz de pesos del modelo en esa iteración.

En un esquema de paralelización del entrenamiento por datos, existe el riesgo de incurrir en *overfitting* sobre las particiones si los pesos no se sincronizan con la frecuencia suficiente. Por otro lado, si la sincronización se realiza con demasiada frecuencia, el tiempo de comunicación entre los nodos puede volverse dominante. Este costo no es en absoluto despreciable: si la comunicación representa una fracción significativa del tiempo total de entrenamiento, la distribución del cómputo pierde sentido, ya que la ejecución paralela termina siendo más lenta que el entrenamiento secuencial.

Adicionalmente, no alcanza con dejar que cada nodo optimice por separado hasta una solución especializada sobre su partición y luego combinar esos modelos mediante un promedio de sus pesos, ya que el resultado de cada iteración de los algoritmos de optimización que se utilizan para el entrenamiento depende del anterior y esas soluciones especializadas no son directamente combinables. Por eso las estrategias distribuidas sincronizan durante el entrenamiento, agregando los gradientes de los nodos en cada paso en lugar de promediar modelos ya entrenados por separado.

Este equilibrio entre convergencia y eficiencia de comunicación es, en definitiva, el problema que este trabajo aborda. Ahora bien, sobre ese problema general se recorta una dificultad más concreta, que es la que motivó la construcción del sistema. La literatura describe las estrategias de distribución y reporta resultados sobre ellas, pero comparar esas estrategias entre sí de forma controlada es difícil. Las implementaciones de referencia disponibles están optimizadas para escenarios de producción, arrastran décadas de trabajo de ingeniería que no es uniforme entre estrategias y ocultan detrás de capas de abstracción exactamente aquellas decisiones que se querría poder variar: el régimen de sincronización, la ubicación del optimizador, el esquema de particionado de los parámetros, la codificación de los mensajes. Al comparar dos estrategias sobre dos implementaciones distintas, la diferencia medida mezcla el efecto del algoritmo con el de su calidad de implementación.

Se detectó entonces la ausencia de una base común, parametrizable y de código abierto, sobre la cual las distintas estrategias de distribución compartan la misma capa de comunicación, el mismo motor de redes neuronales y el mismo plano de control, de modo que la única variable entre dos ejecuciones sea la estrategia bajo estudio. Sin esa base, cualquier comparación entre algoritmos, y en particular la evaluación de mejoras propuestas sobre ellos, queda expuesta a un sesgo que no se puede cuantificar. Construir esa base, e instrumentarla para que produzca mediciones, es el problema concreto que este trabajo resuelve.

8 Solución implementada

Se construyó Oxidized Neural Orchestra (O.N.O.), un sistema distribuido de entrenamiento de modelos de aprendizaje profundo. Comprende una biblioteca de redes neuronales, una capa de comunicación con protocolo propio, un plano de control que coordina la ejecución, los tres algoritmos de distribución y tres interfaces de uso.

Esta sección describe la arquitectura resultante. El énfasis está puesto en las decisiones de diseño y en las razones que las motivaron, por encima de los detalles de implementación: en un trabajo cuyo objetivo era construir una base de comparación, son esas decisiones las que determinan si la base sirve para lo que fue construida.

8.1 Visión general

El sistema se organiza como un *workspace* de Cargo con ocho crates y unas 27 000 líneas de código, más una suite de experimentación en Python. La dependencia entre ellos es estrictamente jerárquica: `comms` no depende de ningún otro crate del sistema, `machine_learning` depende únicamente de `comms` y no contiene nada relativo a redes, y sobre ambos se construyen los runtimes de los nodos y el plano de control.

Cuadro 1: Crates del workspace y su responsabilidad.

Crate	Rol
<code>comms</code>	Capa de red: protocolo, transporte, handles tipados, compresión, reparto del dataset
<code>machine_learning</code>	Motor de redes neuronales, sin networking
<code>parameter_server</code>	Runtime del servidor de parámetros: almacenamiento y sincronización
<code>worker</code>	Runtime del nodo trabajador en sus dos variantes, más los middlewares de topología
<code>orchestrator</code>	Esquema de configuración, validación, cálculo de topología y sesión de entrenamiento
<code>node</code>	El único binario desplegable, agnóstico del rol
<code>orchestui</code>	Interfaz interactiva de terminal
<code>orchestra-py</code>	Biblioteca de Python sobre la interfaz de funciones externas

La decisión rectora de la arquitectura es el agnosticismo de rol. Todas las máquinas del clúster ejecutan el mismo binario, `node`; el rol que cada una cumplirá, trabajador o servidor de parámetros, se determina en tiempo de ejecución a partir de la especificación que el orquestador le entrega al conectarse. La configuración de un entrenamiento lleva una única lista plana de direcciones más la cantidad de servidores deseada, y nunca declara qué dirección será servidor: eso lo decide el orquestador a partir de las latencias que mide.

La alternativa evidente era mantener binarios separados de trabajador y de servidor. Se eligió la unificación porque el sistema debía poder cambiar la topología en tiempo de ejecución: Strategy-

Switch requiere que un nodo que arrancó como trabajador se convierta en servidor a mitad del entrenamiento, y eso es imposible si el rol está fijado en el binario desplegado. El costo fue obligar a usar tareas de ejecución local en el runtime asincrónico, porque uno de los runtimes contiene un objeto de entrenamiento que no puede moverse entre hilos.

La segunda decisión estructural es la separación entre infraestructura y dominio. El trabajador no sabe nada de redes neuronales: es genérico sobre una estrategia de entrenamiento, y todo lo relativo al aprendizaje vive detrás de contratos compartidos. Del lado de la red, la contracara de esa regla es que los trucos de codificación y de precisión viven exclusivamente en **comms**. Los trabajadores y los servidores nunca tocan media precisión y siempre operan en precisión simple; la conversión ocurre únicamente en el serializador y el deserializador.

8.2 Capa de comunicación

8.2.1 Protocolo de dos planos

El protocolo separa el plano de control del plano de datos y los serializa de manera distinta, porque las exigencias de uno y otro son opuestas.

El plano de control transporta veinte variantes de comando, entre ellas el establecimiento de conexión, la creación de nodo con su especificación, la medición de latencia, el reporte de pérdidas, la orden de detención y la promoción de un nodo a servidor. Son mensajes poco frecuentes y estructuralmente ricos, de modo que se serializan como JSON: la ergonomía y la evolucionabilidad del formato pesan más que los bytes.

El plano de datos transporta gradientes, parámetros y fragmentos del conjunto de datos. Son mensajes grandes y frecuentes, y se serializan reinterpretando la memoria directamente, sin ninguna biblioteca de serialización de por medio. El resultado es un camino de datos sin copias, y esa propiedad se apoya en tres detalles concretos. El envío emite una única llamada al sistema vectorizada sobre el encabezado y el tensor, de modo que el tensor nunca se copia a un buffer intermedio. El buffer de recepción está respaldado por un vector de enteros de 32 bits en lugar de bytes, para garantizar el alineamiento a cuatro bytes que la reinterpretación de memoria requiere. Y el mensaje de parámetros expone una referencia mutable, lo que permite al receptor modificarlos en el propio buffer de lectura sin copiarlos.

8.2.2 Transporte y reducción del tráfico

El transporte se define como un *trait* asincrónico con dos operaciones, envío y recepción, pensado para apilarse siguiendo el patrón decorador. Al cierre del trabajo tiene una única capa, la de enmarcado de mensajes, de modo que el sistema hereda su fiabilidad de TCP. La discusión sobre las capas de reintento y vencimiento que llegó a tener, y las razones por las que se retiraron, se retoma en *Riesgos materializados y lecciones aprendidas*.

Sobre ese transporte se aplican dos técnicas complementarias de reducción del volumen transmitido. La primera es la codificación de gradientes en media precisión, que reduce a la mitad los bytes enviados; se aplica únicamente a los gradientes, mientras que los parámetros y los fragmentos de datos viajan en precisión simple.

La segunda es un protocolo de gradientes dispersos que sigue el enfoque de Aji y Heafield (Aji & Heafield, 2017): se transmiten únicamente los componentes de mayor magnitud del gradiente y los descartados se acumulan en un residuo que se considera en el mensaje siguiente. El umbral que separa unos de otros no se calcula de forma exacta, ya que hacerlo exigiría ordenar el gradiente completo en cada paso; se estima muestreando hasta 16 384 valores y aplicando una selección parcial de costo lineal. El formato serializado codifica tramos contiguos por encima del umbral mediante desplazamientos relativos, lo que evita transmitir un índice por cada valor.

La elección entre ambas es adaptativa: se calcula la versión dispersa y solo se utiliza si su tamaño resulta efectivamente menor que el de la versión densa en media precisión; en caso contrario se degrada a esta última. Un gradiente denso, que es lo que produce una red convolucional, no se beneficia de la codificación dispersa, y el sistema lo detecta en lugar de suponerlo.

8.2.3 Handles tipados

Toda la capa de comunicación usa el sistema de tipos para hacer irrepresentables los estados inválidos. Cada rol tiene su propio handle con únicamente las operaciones que le corresponden, y la asignación de rol a un nodo recién conectado consume el handle sin rol y devuelve uno de trabajador o de servidor, lo que hace imposible por construcción operar sobre un nodo al que todavía no se le asignó un papel. El mismo mecanismo sostiene la promoción en Strategy-Switch, donde el handle de trabajador se consume y devuelve un handle de servidor reutilizando la misma conexión, de modo que la transición no puede dejar al sistema en un estado intermedio. La misma idea se aplica a los hiperparámetros con rangos acotados, que se representan mediante tipos que validan en el borde de deserialización.

8.3 Motor de redes neuronales

El motor se construyó sobre `ndarray`, que provee las operaciones sobre arreglos numéricos n -dimensionales y la representación de los tensores. La propagación hacia adelante y hacia atrás, los optimizadores, las inicializaciones y las funciones de pérdida se implementaron en el marco de este trabajo, sin recurrir a ningún framework de aprendizaje profundo.

8.3.1 El modelo como buffer plano

La decisión de diseño más consecuente del motor es que las capas no almacenan sus parámetros. Los parámetros se pasan como un fragmento plano de memoria en cada llamada, y la capa los reinterpreta con la forma que le corresponde. Las capas sí conservan estado de trabajo, que es la entrada de la pasada y los buffers precomputados para la propagación hacia atrás. Ese estado se dimensiona durante la primera época y luego se reutiliza, de modo que después de ese calentamiento una pasada de entrenamiento no reserva memoria dinámica.

El modelo completo vive entonces en un único buffer plano, sobre el cual las capas operan como vistas definidas por desplazamientos y formas. Esa representación habilita tres cosas que de otro modo serían imposibles: que la entrada y salida de red no requiera copias, ya que el buffer del modelo es directamente el buffer que se transmite; que el modelo pueda partirse entre varios servidores entregándole a cada uno un rango del buffer; y que la misma implementación funcione sin cambios tanto cuando una sola entidad posee todas las capas como cuando están repartidas. El componente

que unifica los tres algoritmos no serializa nada: rebana uno o varios buffers planos y entrega a cada capa su ventana, guiado bajo Parameter Server por un vector de asignación que indica a qué servidor pertenece cada capa. El mismo modelo secuencial corre sin modificaciones en ambos regímenes, y es el punto de polimorfismo entre algoritmos del sistema.

8.3.2 Componentes

Las capas se modelan como un enumerado cerrado con despacho estático y no como un *trait* con despacho dinámico, decisión coherente con el objetivo de rendimiento. Se implementaron capas densas, convolucionales, de *max-pooling*, de reordenamiento de forma y las activaciones sigmoide, tangente hiperbólica, ReLU y softmax. La capa de reordenamiento, que es de costo nulo, es el adaptador que permite convivir a las capas densas con las convolucionales, y el constructor la inserta automáticamente en las fronteras correspondientes, de modo que el lenguaje de configuración no necesita exponer el concepto.

La convolución concentró buena parte del esfuerzo del trabajo. Su implementación final se basa en reformular la convolución como una multiplicación de matrices, lo que permite delegar el trabajo pesado en las rutinas de álgebra lineal optimizadas y aprovechar la localidad de caché. Se llegó a ella después de probar y descartar dos alternativas, la transformada rápida de Fourier y la paralelización con hilos, según se detalla en las lecciones aprendidas. La mejora fue de 2,27 veces en el tiempo total de entrenamiento.

Se implementaron dos funciones de pérdida, error cuadrático medio y entropía cruzada, que calculan el valor y el gradiente en una misma pasada. La entropía cruzada acota su argumento para protegerse del logaritmo de cero.

Los optimizadores implementados son descenso por gradiente, descenso con momento clásico y Adam (Kingma & Ba, 2014; Sutskever, Martens, Dahl, & Hinton, 2013). Las inicializaciones cubren los esquemas de Xavier-Glorot (Glorot & Bengio, 2010), Kaiming-He (He, Zhang, Ren, & Sun, 2015) y LeCun, en sus variantes uniforme y normal. Los inicializadores no son generadores por tensor sino generadores con presupuesto que producen valores sobre demanda, lo cual es consecuencia natural de que el modelo sea un buffer plano; un mecanismo adicional resuelve los pedidos que cruzan la frontera entre dos generadores, y es el que permite inicializar cada capa con su esquema propio sobre una única tira de memoria.

La ubicación del optimizador merece explicarse, porque es una decisión de diseño y no un detalle. En el bucle interno, cada worker calcula el gradiente de un mini-batch y lo aplica localmente mediante descenso por gradiente simple, sin estado. El optimizador configurado, que puede tener estado, se aplica una sola vez por paso de sincronización, cuando ya están agregados los gradientes de todos los workers: en el servidor bajo Parameter Server, y en cada nodo bajo All-Reduce. La razón es que un optimizador con estado en el bucle interno acumularía momentos contra una posición del modelo que la sincronización va a sobrescribir, dejando ese estado inconsistente con el punto en el que el modelo efectivamente queda tras agregar.

El modelo entrenado se exporta en formato **safetensors**, siguiendo la convención de nombres de PyTorch para facilitar la interoperabilidad. El layout interno de las capas densas es la traspuesta del que usa PyTorch, de modo que un cargador debe transponer.

8.4 Los tres algoritmos

8.4.1 Parameter Server

El sistema implementa un Parameter Server multi-servidor con particionado del modelo. La decisión de diseño relevante es cómo se reparten las capas entre los servidores.

El particionado equitativo por cantidad de parámetros deja los parámetros de una misma capa repartidos entre dos servidores, con lo cual el fragmento deja de ser contiguo y fuerza una reserva de memoria. Se optó por particionar respetando los límites de capa, repartiendo capas completas entre los servidores mediante un algoritmo *greedy* de empaquetado que asigna primero las más grandes. Junto con la asignación viaja al trabajador un vector de orden que le indica, para cada capa, a qué servidor debe pedirle su fragmento.

Esa decisión tiene una consecuencia en el reensamblado final del modelo: como las capas se reparten por tamaño y no por orden, concatenar ingenuamente lo que devuelve cada servidor produciría un modelo barajado, de modo que el reensamblado recorre explícitamente los desplazamientos por capa.

Del lado del servidor, el almacenamiento y la sincronización se abstraen detrás de dos *traits* independientes, lo que convierte el régimen de operación en una elección de configuración en tiempo de ejecución en lugar de una decisión de compilación. El almacenamiento con bloqueo está fragmentado internamente y usa doble buffer: los trabajadores siguen acumulando sobre el buffer activo mientras el congelado se consume, y la actualización usa una operación atómica de comparación e intercambio que la vuelve de ejecución única, de modo que si otro hilo ya está actualizando, la llamada no hace nada.

El sincronizador con barrera es el que da la semántica sincrónica utilizada en los experimentos. Su contador es dinámico: un trabajador que termina su entrenamiento y se retira lo decrementa permanentemente, y si con eso la barrera se completa, la avanza él mismo antes de irse. Esa lógica está atada a la destrucción del objeto, de modo que ocurre necesariamente, y resuelve el problema de membresía estática frente a terminación dinámica que se discute en las lecciones aprendidas.

La segunda variante de almacenamiento sigue el enfoque sin bloqueos de *Hogwild!* (Niu et al., 2011). Está implementada, pero no se utilizó en los benchmarks por las razones de corrección y de validez de supuestos que se detallan en las lecciones aprendidas.

8.4.2 Ring All-Reduce

La implementación sigue la formulación en anillo de Patarasuk y Yuan (Patarasuk & Yuan, 2009), en dos fases de $n - 1$ pasos cada una. En la primera, cada nodo termina con un fragmento del gradiente reducido sobre todos los participantes; en la segunda, ese fragmento ya reducido se propaga por el anillo. El costo total es de $2(n - 1)$ pasos, cada uno moviendo aproximadamente $1/n$ del modelo, lo que da el costo óptimo en ancho de banda.

Tres decisiones concretas la sostienen. El fragmentado es balanceado: los fragmentos difieren a lo sumo en un elemento, en lugar de dejar una cola corta, de modo que ningún paso del anillo es sistemáticamente más lento que los demás. El envío y la recepción de cada paso son concurrentes, y lo mismo vale para la construcción del anillo, donde aceptar la conexión entrante y establecer la saliente también se hacen en paralelo; hacerlo de forma secuencial provoca que todos los nodos intenten enviar antes de recibir y los buffers de los sockets se llenen sin que nadie los drene. Y el

gradiente reducido se promedia dividiendo por la cantidad de trabajadores, de modo que la tasa de aprendizaje efectiva no escale con el tamaño del anillo.

8.4.3 Strategy Switch

La implementación de Strategy-Switch (Provatas et al., 2025) separa la decisión, que vive en el orquestador, de la ejecución, que ocurre en los nodos. El criterio de conmutación mantiene una ventana deslizante de las últimas pérdidas y calcula su cambio relativo medio; cuando ese valor cae por debajo de un umbral, es decir, cuando la curva de pérdida se aplana, se dispara la transición. La ventana y el umbral son los valores indicados en el trabajo original.

El plan se precomputa antes de entrenar. La topología completa del Parameter Server, la ubicación de los servidores y el particionado de las capas se calculan en tiempo de configuración, y para cada nodo se construye por anticipado la acción que le corresponderá; lo único dinámico es el momento en que se ejecuta. La razón es que calcular la topología requiere las mediciones de latencia, que ya se hicieron al conectar, y hacerlo en caliente introduciría una pausa en el entrenamiento.

La promoción de un trabajador a servidor transfiere los parámetros ya entrenados: el nodo rebana sus pesos según los rangos recibidos y arranca como servidor sembrado con ellos, no con valores aleatorios. La ligadura entre cada promoción y su fragmento se hace por dirección del servidor y no por orden de aparición en el recorrido de la topología.

La transición es unidireccional, de All-Reduce a Parameter Server, y de un solo disparo por sesión.

8.5 Plano de control

El orquestador es el plano de control del sistema. Recibe la configuración del entrenamiento, la valida, se conecta a los nodos, mide la topología de la red, deriva de allí la asignación de roles y especificaciones, y conduce la sesión.

8.5.1 Selección de topología

Una de las líneas no previstas en el plan original, y de las más interesantes, es la selección de topología a partir de mediciones. Antes de comenzar, el orquestador realiza rondas de medición de latencia entre todos los pares de nodos y construye con ellas un grafo, cuyas aristas se pesan con el RTT máximo observado y no con el promedio. Ambos problemas de optimización se plantean, en consecuencia, sobre el peor caso. El criterio quedó resumido en la discusión que lo originó: *“es mejor tener 2 conexiones más o menos que una muy buena y una muy mala”*, que es exactamente la diferencia entre optimizar la suma y optimizar el máximo.

El orden del anillo de All-Reduce se resuelve como un problema del viajante de comercio, mediante programación dinámica sobre subconjuntos. La ubicación de los servidores de parámetros se resuelve por *backtracking* sobre las combinaciones de k servidores: en Parameter Server los nodos forman un grafo bipartito completo $K_{n,m}$ con n trabajadores y m servidores, de modo que todas las conexiones son entre trabajadores y servidores, y se elige la combinación cuya conexión máxima resulte la menor.

Ambos son algoritmos exactos y no heurísticas, lo cual es defendible dado el régimen al que apunta el sistema, de decenas de nodos, aunque el del ciclo degenera para tamaños grandes.

8.5.2 Sesión orientada a eventos

La ejecución de un entrenamiento se modela como una sesión orientada a eventos. El orquestador expone un canal de eventos y mantiene una tarea por trabajador que traduce los mensajes del protocolo en eventos de dominio: pérdidas publicadas, trabajador terminado, entrenamiento completo, nodo promovándose. Ese diseño es lo que permite que las tres interfaces del sistema consuman exactamente la misma información sin duplicar lógica.

El punto de agregación es el manejo de pérdidas: se registra la última pérdida de cada trabajador, se exige que todos los trabajadores vivos hayan reportado, y recién entonces se alimentan los dos criterios que observan la curva, el de detención temprana y el de conmutación de estrategia. Que la condición sea sobre los trabajadores vivos y no sobre una cantidad fija es, otra vez, consecuencia de la membresía dinámica.

La detención temprana (Prechelt, 1998) se implementó como un criterio de tolerancia: cuando tres pérdidas agregadas consecutivas se mueven por debajo de un umbral configurable, se considera que el entrenamiento convergió y se difunde la orden de detención.

8.6 Interfaces

El sistema expone tres formas de uso, todas sobre el mismo plano de control y compartiendo el mismo esquema de configuración canónico.

La interfaz interactiva de terminal permite configurar, lanzar y monitorear un entrenamiento en vivo. Presenta un asistente de configuración con sugerencias y validación, y un panel que muestra la evolución de la pérdida, el progreso por trabajador y una vista de topología. En esa vista, las partículas que representan el flujo de datos entre nodos se mueven al ritmo real del entrenamiento, siguiendo una media móvil del intervalo entre reportes, y permiten ver cómo un trabajador se convierte en servidor durante una conmutación de estrategia. El entrenamiento corre en un hilo de fondo para que la interfaz nunca se bloquee.

La biblioteca de Python se construyó sobre PyO3, como un crate envoltorio separado que mantiene el orquestador como Rust puro. La alternativa, más limpia en teoría, era anotar los tipos existentes del orquestador con los atributos de PyO3 detrás de una bandera de compilación; se descartó por restricciones del compilador, ya que esos atributos no admiten tipos genéricos ni las variantes complejas de enumerados que el esquema de configuración utiliza. El entrenamiento libera el bloqueo global del intérprete y se ejecuta en un hilo dedicado, de modo que el proceso de Python actúa efectivamente como orquestador.

El binario de ejecución desatendida completa el conjunto: no presenta interfaz interactiva, toma un archivo de configuración y ejecuta el entrenamiento hasta el final. Es el que utiliza la suite de benchmarks.

8.7 Estado de la implementación

El sistema compila sin advertencias y su conjunto de pruebas pasa con 87 pruebas en verde. El motor de aprendizaje propio converge a 0,983 de exactitud sobre MNIST con la red de referencia y a 0,980 con LeNet-5, frente a 0,989 y 0,990 de una implementación de PyTorch con la misma receta de

hiperparámetros, lo que sitúa la implementación propia a alrededor de medio punto porcentual de un framework maduro. Los defectos conocidos al cierre y las líneas que quedaron abiertas se detallan en *Trabajos futuros*.

9 Metodología aplicada

Esta sección actualiza lo planteado en la propuesta, contrastando lo previsto con lo efectivamente ejecutado.

9.1 Gestión y roles

Se estableció un compromiso de 500 horas por estudiante a lo largo de dos cuatrimestres, lo que representa un promedio de unas 16 horas semanales por persona sobre 32 semanas. El Ing. Ricardo A. Veiga cumplió la función de tutor y el Dr. Ing. J. Ignacio Alvarez-Hamelin la de co-tutor y orientador en el campo de desempeño, en el marco del CoNexDat, Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería.

Los tres estudiantes participaron de todas las etapas de análisis, implementación y pruebas, y cada integrante concentró su foco principal en un conjunto de componentes, según se detalla en *Aportes individuales*. Esa especialización, que emergió del desarrollo y no estaba impuesta por la propuesta, tuvo un efecto secundario relevante: la revisión cruzada de código se volvió el principal mecanismo de difusión de conocimiento entre componentes, y en varios casos las discusiones de revisión terminaron modificando decisiones de diseño y no solo detalles de implementación.

La coordinación se sostuvo en dos planos. Con los tutores se mantuvieron reuniones periódicas en formato virtual, con la cadencia comprometida en el Acta de Acuerdo de una periodicidad igual o menor a las dos semanas, para informar el avance, definir prioridades y planificar la etapa siguiente. Entre los integrantes, la coordinación fue continua y de cadencia diaria, apoyada en un canal de mensajería para las decisiones rápidas y en las *issues* del repositorio para las que requerían registro.

9.2 Proceso de desarrollo

El método de desarrollo fue incremental, organizado en iteraciones con entregables intermedios. La unidad de entrega fue la *pull request*: cada funcionalidad se desarrolló en una rama propia que se integraba a la rama principal previa revisión. A lo largo del proyecto se produjeron 989 commits, 108 pull requests y 80 issues, entre septiembre de 2025 y julio de 2026.

El código y todos los artefactos versionables se gestionaron con Git sobre un repositorio alojado en GitHub, y los mensajes de los commits siguen la convención de *Conventional Commits*. Los entregables intermedios que marcaron el avance fueron, en orden: la capa de comunicación funcionando de punta a punta, el primer entrenamiento distribuido completo con Parameter Server, el motor de redes neuronales convergiendo sobre MNIST, la interfaz de terminal mostrando un entrenamiento en vivo, All-Reduce en anillo, y finalmente la suite de benchmarks con sus resultados.

9.3 Seguimiento, tickets y gestión del alcance

El seguimiento del proceso, la gestión de tickets y el registro de errores se realizaron con las *issues* del repositorio, categorizadas mediante etiquetas por tipo (*bug*, *enhancement*, *documentation*) y por componente del sistema (*parameter-server*, *comms*, *worker*, *ffi*, *tui*, entre otras). Las issues se usaron además, y esto excedió lo previsto en la propuesta, como espacio de debate de diseño: varias de las decisiones de arquitectura documentadas en este informe se resolvieron enteramente en el hilo de

comentarios de una issue, con enumeración de alternativas y su descarte razonado. Ese registro fue el insumo principal para reconstruir el proceso.

Los errores se clasificaron por criticidad, distinguiendo los que impedían el avance del trabajo de los que admitían corrección diferida. La gestión del alcance se materializó en dos momentos explícitos: el diferimiento de las funcionalidades no esenciales hasta tener un producto mínimo funcionando, tomado al inicio bajo el criterio de que *“estas features van al final, una vez que tengamos el MVP andando”*, y una sesión de definición de alcance previa al cierre en la que se descartaron de forma deliberada las líneas que no iban a llegar. Ambos se detallan en la sección de riesgos.

9.4 Automatización

La construcción y las pruebas del sistema se ejecutan con las herramientas del ecosistema de Rust (`cargo build`, `cargo test`, `cargo clippy` para análisis estático), y los entornos de simulación se levantan de forma automatizada mediante Docker y un `Makefile` autodocumentado, lo que hace reproducible el despliegue de las distintas configuraciones de nodos. Para el código Python se incorporó `ruff` como analizador estático y formateador.

La construcción, las pruebas, el análisis estático, el levantamiento de entornos multinodo y la ejecución de benchmarks completos quedaron totalmente automatizados y de ejecución desatendida: el conjunto final de 38 configuraciones se ejecutó de un solo comando durante 5 horas y 39 minutos sin intervención. La integración continua, en cambio, no llegó a funcionar: la propuesta ya la señalaba como una cuestión no definida cuya adopción dependería de la evolución del proyecto, y el flujo de trabajo que se escribió nunca se ejecutó por un error en la ruta del directorio que lo aloja, detectado en la limpieza final del repositorio. La verificación previa a la integración quedó, en los hechos, a cargo de la ejecución local de las pruebas por parte de quien abría la pull request y de quien la revisaba.

9.5 Criterios de aceptación

Cada pull request se sometió a revisión de código por parte de otro integrante antes de integrarse. Una entrega se consideró aceptada cuando se cumplían de forma conjunta tres condiciones: la revisión de código fue aprobada, la totalidad de las pruebas automatizadas asociadas se ejecutaba con éxito, y la funcionalidad requerida quedaba demostrada mediante una prueba de aceptación. Para las funcionalidades que involucran coordinación entre nodos, esa prueba consistió en una ejecución completa de entrenamiento distribuido sobre el entorno simulado, verificando que la convergencia obtenida fuera consistente con la del entrenamiento secuencial equivalente.

El criterio funcionó para las fallas visibles, pero resultó insuficiente para detectar errores de corrección distribuida silenciosos, aquellos que no provocan fallos sino que degradan la calidad del modelo. La forma en que finalmente aparecieron se detalla en las lecciones aprendidas.

9.6 Artefactos e indicadores

Como artefactos de gestión se llevaron minutas de las reuniones periódicas, el registro del alcance y el registro de riesgos, revisado en las reuniones de seguimiento. A ellos se sumaron dos artefactos

técnicos no previstos que resultaron centrales: el documento canónico del esquema de configuración y el borrador de arquitectura, que sostuvo las discusiones de diseño en las etapas iniciales.

Sobre la calidad del proceso se definieron tres indicadores: el tiempo entre apertura y cierre de una issue, la proporción de pull requests que requieren correcciones tras la revisión, y el desvío entre horas planificadas y efectivamente dedicadas. Este último funcionó también como indicador de costos, por las razones que se detallan en la evaluación de impacto económico.

Sobre la calidad del producto se consideraron la proporción de pruebas exitosas sobre el total y la cantidad de advertencias del análisis estático, esta última mantenida en cero mediante `clippy` de forma sostenida. La cobertura de pruebas, en cambio, se relevó de manera irregular y no se sostuvo como indicador continuo, con consecuencias verificables: la variante de actualización sin bloqueos de los parámetros nunca tuvo cobertura y el error que la invalidaba permaneció oculto hasta una auditoría tardía, y un error en la propagación hacia atrás del *max-pooling* quedó enmascarado porque las pruebas existentes usaban un único canal y un único elemento de batch.

10 Herramientas externas utilizadas

En cumplimiento de las pautas de ética profesional, se declaran a continuación todas las herramientas empleadas en el desarrollo del trabajo, junto con las consideraciones que motivaron su elección.

La propuesta fijó tres tecnologías desde el inicio, Rust, Python y Docker, y dejó explícitamente abiertas las decisiones restantes, estableciendo tres criterios para tomarlas: que no introdujeran un costo de comunicación o de serialización capaz de distorsionar la comparación entre estrategias, que se integraran con el sistema de construcción y de pruebas del ecosistema de Rust sin requerir dependencias externas a él, y que su licencia fuera de código abierto de modo que el sistema pudiera ser retomado por terceros. Los tres criterios se sostuvieron.

10.1 Lenguajes y entornos

Rust. Se eligió como lenguaje del sistema principal por tres motivos. El primero es el control de bajo nivel que ofrece sin renunciar a garantías de seguridad, lo que resultó determinante para un sistema cuyo camino crítico exige manipular memoria sin copias. El segundo es la *conurrencia sin miedo*: el compilador verifica estáticamente una clase completa de errores de concurrencia. El tercero es la relevancia creciente del lenguaje en el ámbito de los sistemas. La evaluación a posteriori de esa elección, y el alcance exacto de lo que sus garantías cubren, se retoma en las lecciones aprendidas.

El ecosistema se utilizó completo: **cargo** para construcción y gestión de dependencias, **cargo test** como arnés de pruebas, **cargo clippy** para análisis estático y **cargo fmt** para formato. No se incorporó ninguna herramienta externa a él, según el criterio fijado en la propuesta.

Python. Se utilizó para tres propósitos: la interfaz de funciones externas del sistema, por ser el lenguaje dominante en aprendizaje profundo; la suite de experimentación y el análisis de resultados; y los scripts de generación y despliegue del entorno contenedizado. El código Python se verifica con **ruff** como analizador estático y formateador.

Docker. Se utilizó para simular la ejecución del sistema sobre múltiples nodos y para automatizar el despliegue. Se acompaña de **cargo-chef** para cachear las capas de dependencias en la construcción de la imagen, lo que reduce de forma sustancial el tiempo de reconstrucción.

Git y GitHub. Control de versiones y gestión del proceso: ramas por funcionalidad, *pull requests* con revisión obligatoria, e *issues* para el seguimiento de tickets, errores y debates de diseño.

10.2 Bibliotecas de Rust

La elección de cada una respondió a los criterios enunciados. Todas son de código abierto.

Biblioteca	Uso
<code>tokio</code>	Runtime asincrónico, TCP, tareas y canales. Base de todo el sistema distribuido
<code>ndarray</code>	Operaciones sobre arreglos numéricos n-dimensionales; provee la multiplicación de matrices del motor
<code>ndarray-rand</code>	Inicialización aleatoria de tensores

Biblioteca	Uso
<code>rayon</code>	Paralelismo de datos en el almacenamiento del servidor de parámetros
<code>serde</code> y <code>serde_json</code>	Serialización del plano de control y de los archivos de configuración
<code>bytemuck</code>	Reinterpretación de memoria sin copias entre valores numéricos y bytes
<code>half</code>	Tipo de media precisión para la compresión de gradientes
<code>uuid</code>	Identidad estable de nodos
<code>futures</code>	Composición concurrente de operaciones de red
<code>rand</code> y <code>rand_distr</code>	Muestreo del umbral disperso, barajado y distribuciones de inicialización
<code>parking_lot</code>	Primitivas de sincronización de la barrera dinámica
<code>trait-variant</code> y <code>async-trait</code>	Traits asincrónicos
<code>safetensors</code>	Exportación del modelo entrenado
<code>log</code> y <code>tracing-subscriber</code>	Registro estructurado con filtrado por variable de entorno
<code>ratatui</code> y <code>crossterm</code>	Interfaz interactiva de terminal
<code>pyo3</code>	Interfaz de funciones externas con Python
<code>approx</code>	Comparaciones aproximadas en las pruebas
<code>anyhow</code>	Manejo de errores en la interfaz de terminal

Tres de estas elecciones merecen justificarse porque tenían alternativas reales.

ndarray en lugar de un framework de aprendizaje profundo. Es la decisión que define el alcance del trabajo. Se eligió una biblioteca que provee *únicamente* operaciones sobre arreglos numéricos, sin diferenciación automática, sin capas y sin optimizadores, porque el objetivo del trabajo incluía implementar el motor de aprendizaje. Usar un framework existente habría hecho el trabajo considerablemente más simple y considerablemente menos valioso.

bytemuck para la serialización del plano de datos. Aquí operó de forma directa el primer criterio de la propuesta. Una biblioteca de serialización de propósito general habría introducido una copia y un costo de codificación en el camino crítico, exactamente el tipo de sobrecarga que podría distorsionar la comparación entre estrategias que comunican de maneras distintas. La reinterpretación de memoria elimina esa variable.

tracing-subscriber en reemplazo de env_logger. Se migró tardíamente, manteniendo la fachada genérica de registro en las bibliotecas. La razón fue concreta: la interfaz de terminal se corrompía con salidas de registro dirigidas a la salida estándar, y el registro debía dirigirse a la salida de error de forma consistente.

10.3 Bibliotecas de Python

Biblioteca	Uso
<code>maturin</code>	Construcción de la extensión nativa que expone el sistema a Python
<code>numpy</code>	Lectura de los conjuntos de datos binarios en la evaluación
<code>matplotlib</code>	Generación de todas las figuras de los experimentos
<code>safetensors</code>	Carga de los pesos entrenados por el sistema para su evaluación
<code>torch</code>	Línea de base de referencia
<code>ruff</code>	Análisis estático y formato

El uso de PyTorch requiere una aclaración, porque el trabajo declara no apoyarse en frameworks de aprendizaje profundo. PyTorch no participa del sistema en ninguna forma: no se utiliza para entrenar, ni para calcular gradientes, ni para representar modelos. Se lo emplea exclusivamente como línea de base externa, entrenando en un solo proceso el mismo modelo con la misma receta de hiperparámetros, para contrastar la exactitud que alcanza el motor propio. Su presencia en el repositorio es una dependencia opcional del entorno de experimentación, separada de las dependencias de ejecución del sistema.

10.4 Documentación y edición del informe

El presente informe y la propuesta que lo precedió se escriben en Markdown y se componen a PDF mediante **Pandoc**, con procesamiento de citas y bibliografía en formato BibTeX bajo el estilo APA 6. Las monografías que dieron origen a los estudios experimentales se compusieron en LaTeX con la clase de artículo de IEEE. Los diagramas se generaron con **Mermaid** y **Matplotlib**.

10.5 Herramientas de inteligencia artificial generativa

En cumplimiento de la obligación de declararlo explícitamente, se deja constancia de que durante el desarrollo del trabajo se utilizaron **modelos grandes de lenguaje** como asistencia, en los siguientes usos: consulta y discusión de conceptos de sistemas distribuidos y de aprendizaje profundo, asistencia en la depuración de errores, revisión y sugerencia de mejoras sobre código ya escrito, generación de código auxiliar en tareas accesorias como scripts de despliegue y de graficado, y asistencia en la redacción y revisión de la documentación y de este informe.

Los integrantes del equipo asumen la responsabilidad plena sobre la totalidad del contenido generado, se apropian del mismo, lo conocen en profundidad y están en condiciones de defenderlo en el acto de Defensa. Delegar la ejecución de una tarea en una herramienta no exime de la responsabilidad sobre su resultado. En particular, se deja constancia de que las decisiones de arquitectura documentadas en este informe, los criterios metodológicos de los estudios experimentales y la interpretación de sus resultados son producto del trabajo y del debate del equipo, y su registro en las *issues* y *pull requests* del repositorio es verificable de forma independiente.

10.6 Consideración sobre licencias

La totalidad de las herramientas y bibliotecas utilizadas son de código abierto, con licencias permisivas (MIT o Apache 2.0 en su mayoría). El sistema desarrollado se publica bajo licencia abierta, de modo que pueda ser retomado y extendido por terceros, según el criterio establecido en la propuesta. No se incurrió en costos de licenciamiento.

11 Experimentación y validación

La validación se apoyó en dos frentes. El primero es el de las pruebas automatizadas, que verifican que el sistema hace lo que dice hacer. El segundo, y el que da sentido al proyecto, es el de los estudios experimentales: como O.N.O. fue concebido para comparar estrategias de distribución de forma controlada, la validación última consiste en demostrar que esa base produce evidencia comparable.

Los estudios se organizan como tres investigaciones independientes, cada una con su pregunta, su metodología y sus conclusiones acotadas al entorno evaluado. Las tres corren sobre la misma base, de modo que la única variable entre configuraciones es aquella que cada estudio se propone aislar.

11.1 Pruebas automatizadas

El sistema se verificó con tres tipos de pruebas. Las pruebas unitarias ejercitan cada componente de forma aislada y son las que sostienen el motor de redes neuronales: la corrección de la propagación hacia atrás no es observable a simple vista, y la única manera práctica de detectar un gradiente mal derivado es contrastarlo contra un valor calculado de forma independiente. Las pruebas de integración validan la interacción entre nodos durante una ejecución distribuida, en particular los protocolos de coordinación. Las pruebas de aceptación comprueban que cada funcionalidad requerida se comporta como fue especificada, ejecutando un entrenamiento completo sobre el entorno simulado y verificando que la convergencia obtenida sea consistente con la del entrenamiento secuencial equivalente.

Las dos primeras se escribieron con el arnés de pruebas nativo de Rust y se ejecutan mediante `cargo test`; las de aceptación se levantan sobre los entornos contenedizados descritos más abajo. Las tres son automatizadas y de ejecución desatendida, y se versionan junto con el código, de modo que cada cambio pueda validarse antes de integrarse.

11.2 Entorno de experimentación y reproducibilidad

El despliegue se realiza con una única imagen de Docker, coherente con el agnosticismo de rol del sistema: no hay imagen de trabajador ni imagen de servidor. Un conjunto de scripts genera la configuración del clúster, ajusta la resolución de nombres y levanta el entorno a partir de un único parámetro con la cantidad de nodos.

Todas las mediciones se realizaron sobre clústeres simulados en una única máquina física: un contenedor por nodo, red *bridge* y puertos publicados. Esta decisión, tomada al inicio del trabajo por la imposibilidad de disponer de múltiples máquinas dedicadas, es la limitación transversal a los tres estudios y condiciona la lectura de todos los resultados de rendimiento. La infraestructura no fija afinidad de núcleos, no limita CPU por contenedor y no emula latencia ni ancho de banda: al aumentar la cantidad de nodos los núcleos físicos se saturan, de modo que lo que se mide es escalado lógico bajo recursos compartidos y no escalado multimáquina. La red, además, es efectivamente loopback, lo que favorece sistemáticamente a las estrategias que comunican más y hace que los tiempos medidos sean una cota inferior del costo de comunicación.

Esta es la única declaración de la limitación en el informe, y los tres estudios se leen bajo ella. Para separar lo que es propiedad de un algoritmo de lo que es artefacto del entorno, los resultados de tiempo se acompañan de la métrica analítica que se introduce en el primer estudio.

La suite de experimentación está escrita en Python y organizada de forma declarativa en cuatro conjuntos de ensayos que miden convergencia, velocidad de ejecución, velocidad de convergencia y escalabilidad. Persiste los resultados de forma incremental, agrega repeticiones en media y desvío, y regenera su propia documentación en cada corrida. Documenta además de forma explícita sus criterios de equidad y sus fuentes de ruido: justifica el presupuesto fijo de nodos, aclara qué significa una época en cada ensayo, y explica por qué ciertas variantes se comparan por exactitud y no por velocidad, dado que una medición de tiempo aislada tiene un ruido cercano al 25 %.

Para que las comparaciones sean reproducibles, cada ejecución se documenta junto con la configuración que la generó, se fija la semilla de inicialización de los parámetros y se conservan tanto los resultados crudos como los procesados. Los conjuntos de datos utilizados se resumen en la Tabla 1.

Cuadro 4: Conjuntos de datos utilizados en la evaluación.

Dataset	Entrenamiento	Test	Forma	Clases
MNIST	60 000	10 000	$28 \times 28 \times 1$	10
FashionMNIST	60 000	10 000	$28 \times 28 \times 1$	10

MNIST (LeCun, Bottou, Bengio, & Haffner, 1998) se emplea como referencia de correctitud y convergencia, por tratarse de un problema bien caracterizado sobre el que cualquier desviación resulta evidente. FashionMNIST (Xiao, Rasul, & Vollgraf, 2017) se emplea como benchmark principal en el primer estudio, por ser una tarea comparable en dimensiones pero considerablemente menos separable, lo que evita que todas las configuraciones saturen en exactitudes indistinguibles.

11.3 Estudio I: Parameter Server frente a All-Reduce

11.3.1 Pregunta e hipótesis

El primer estudio aborda la pregunta que motivó el trabajo desde la propuesta: ¿cuándo conviene cada enfoque, por qué, y qué compromisos aparecen entre convergencia, throughput, escalabilidad y sincronización? No se busca aquí el estado del arte en visión por computadora, sino caracterizar el *sistema* de entrenamiento. Es la pregunta que O.N.O. fue construido para responder, y por eso este estudio es el que valida más directamente la premisa del proyecto.

11.3.2 Fundamento

Con W workers, cada uno procesa un mini-batch local de tamaño b y calcula un gradiente g_w . El gradiente agregado es el promedio

$$\bar{g} = \frac{1}{W} \sum_{w=1}^W g_w,$$

que equivale a un paso de descenso por gradiente con batch efectivo $W \cdot b$, es decir, el tamaño de batch que efectivamente “ve” el optimizador tras agregar a los W workers. Promediar, y no sumar, mantiene la magnitud del gradiente, y por consiguiente la tasa de aprendizaje efectiva, independiente

de W . Esta distinción es central para el diseño experimental: en O.N.O. el `batch_size` es por worker, de modo que, con b constante, agregar workers cambia el batch efectivo.

En el régimen de Parameter Server evaluado, los servidores almacenan los pesos fragmentados entre ellos, y los workers hacen *push* de gradientes y *pull* de parámetros. El régimen de sincronización define el comportamiento: con barrera, todos los workers esperan a que el servidor aplique la actualización. En ring All-Reduce, los W workers se ordenan en anillo y ejecutan dos fases, *scatter-reduce* y *all-gather*; el volumen de gradiente que comunica cada worker es aproximadamente

$$2 \frac{W-1}{W} |g|,$$

esencialmente independiente de W salvo un factor constante, lo que le confiere buena eficiencia de ancho de banda. No hay servidor central ni punto único de contención, pero todos avanzan al ritmo del más lento.

Este estudio utiliza el régimen sincrónico con barrera para Parameter Server. La consecuencia es deliberada y debe tenerse presente al leer los resultados: bajo ese régimen, ambas estrategias aplican la misma regla de actualización, de modo que no cabe esperar una diferencia sistemática de convergencia entre ellas. Lo que se compara, entonces, es el costo de llegar al mismo lugar.

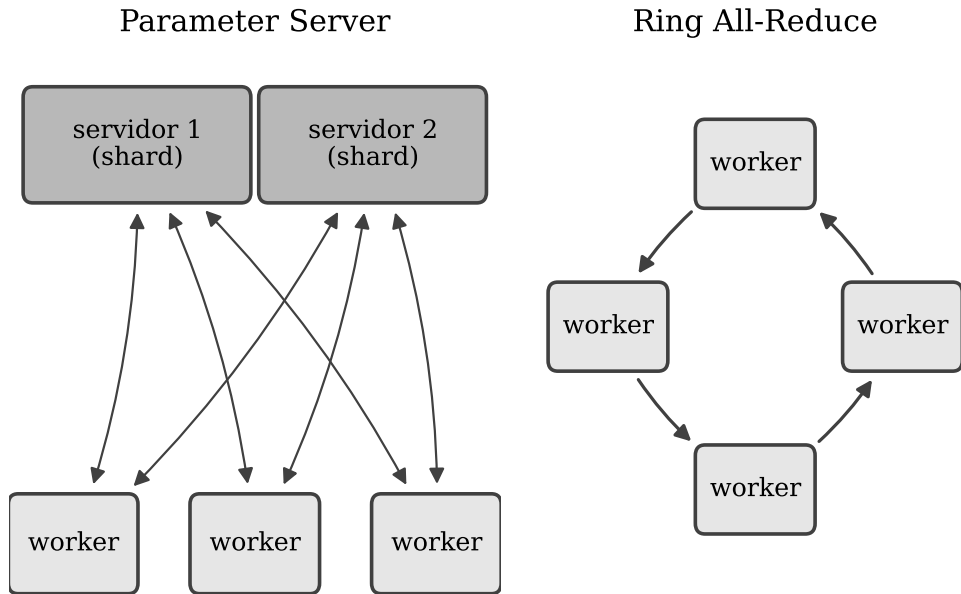


Figura 1: Parameter Server (izquierda): los workers hacen push y pull contra dos servidores que fragmentan los pesos. Ring All-Reduce (derecha): los workers promedian gradientes entre sí sin servidor central.

11.3.3 Criterio de comparación justa

Comparar dos estrategias que no consumen los mismos recursos exige explicitar el criterio de equidad, porque distintos criterios conducen a conclusiones distintas. El criterio adoptado es igual número de

workers: para cada N , All-Reduce usa N workers y Parameter Server usa los mismos N workers más dos servidores. Los servidores se consideran el costo estructural propio de Parameter Server, no un recorte de su capacidad de cómputo.

Este criterio tiene una consecuencia deseable. Como ambas estrategias usan los mismos W workers con el mismo b , el batch efectivo $W \cdot b$ es idéntico para ambas en cada topología. La semántica de optimización queda fijada y, por consiguiente, la convergencia es directamente comparable. El costo del criterio, que se declara abiertamente, es que Parameter Server ocupa más nodos totales para el mismo trabajo, lo cual queda registrado como una desventaja suya en la matriz de decisión final.

Se reportan dos mediciones complementarias, y se es explícito sobre qué pregunta responde cada una: con un batch por worker pequeño se mide convergencia y exactitud; con un batch por worker mayor se mide throughput bruto.

Cuadro 5: Topologías evaluadas bajo el criterio de igual número de workers. Parameter Server emplea dos servidores con parámetros fragmentados. Batch efectivo = workers $\times$ 10.

Estrategia	Workers	Servidores	Nodos	Batch efectivo
AR	3	0	3	30
AR	5	0	5	50
PS	3	2	5	30
PS	5	2	7	50

Los modelos empleados se detallan en la Tabla 3. El modelo principal es **nielsen**, una red convolucional pequeña con softmax y entropía cruzada. Para aislar la presión de comunicación se definieron además redes densas de tamaño creciente cuyo objetivo no es la exactitud sino variar el tamaño del gradiente.

Cuadro 6: Modelos empleados y su tamaño aproximado. Las redes Densa S/M/L son totalmente conectadas.

Modelo	Arquitectura	Parámetros
Nielsen	Conv20@5 $\rightarrow$ Pool $\rightarrow$ Dense100 $\rightarrow$ Dense10	289 630
Densa S	Dense128 $\rightarrow$ Dense10	101 770
Densa M	Dense512 $\rightarrow$ Dense512 $\rightarrow$ Dense10	669 706
Densa L	Dense1024 $\rightarrow$ Dense1024 $\rightarrow$ Dense10	1 863 690

Se utilizó el optimizador de descenso por gradiente estocástico con tasa de aprendizaje 0,1 y semilla fija, sobre una máquina de ocho núcleos.

11.3.4 Resultados: convergencia

Ambas estrategias entrenan correctamente. A igual número de workers, All-Reduce y Parameter Server alcanzan exactitudes de test prácticamente iguales en ambos conjuntos de datos, con Parameter

Server incluso marginalmente por encima en FashionMNIST. Las curvas de pérdida descenden de forma casi solapada.

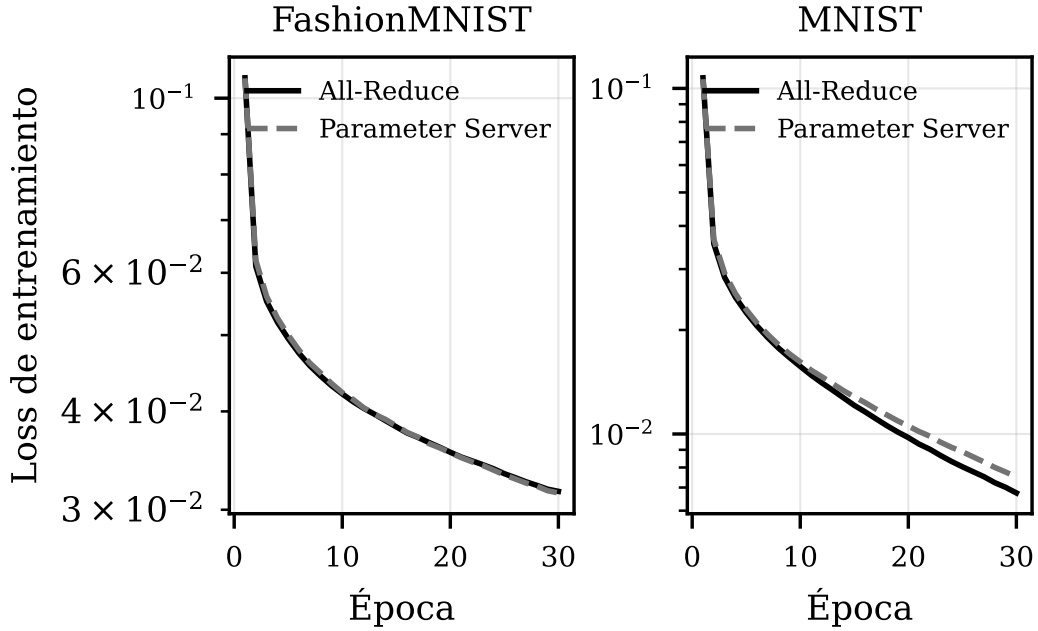


Figura 2: Convergencia (pérdida de entrenamiento por época) a 3 workers, en FashionMNIST (izquierda) y MNIST (derecha), para All-Reduce y Parameter Server.

Este resultado no es una sorpresa sino una confirmación de corrección: como el régimen es sincrónico, ambos esquemas de agregación aplican la misma regla de actualización, y una divergencia sistemática entre ellos habría indicado un error de implementación en alguno de los dos. En ese sentido, esta medición funciona como prueba de aceptación de ambas estrategias tanto como resultado experimental.

Cuadro 7: Resultados de convergencia justa (batch 10 por worker, evaluación sobre el conjunto de test completo) para ambas estrategias en cada conjunto de datos.

Dataset	Estrategia	Workers	Exactitud test (%)	Tiempo total (s)
FashionMNIST	AR	3	87,28	210,9
FashionMNIST	PS	3	87,58	257,6
FashionMNIST	AR	5	86,25	248,1
FashionMNIST	PS	5	86,49	272,5
MNIST	AR	3	97,82	213,8
MNIST	PS	3	97,62	251,5
MNIST	AR	5	97,00	253,4
MNIST	PS	5	96,75	269,1

La diferencia decisiva, entonces, es el tiempo. All-Reduce alcanza la misma exactitud apreciablemente

antes que Parameter Server, con una brecha del orden del 15 % a 20 % en las configuraciones de 3 workers. La explicación es estructural: el servidor de Parameter Server actúa como punto de serialización por el que pasan todas las actualizaciones.

11.3.5 Resultados: throughput y escalabilidad

Con batch por worker fijo, el comportamiento al sumar nodos revela un cruce que matiza la conclusión anterior. Con pocos workers, All-Reduce sostiene mayor throughput; al sumar workers, las tendencias se invierten: el throughput de All-Reduce cae levemente, porque su anillo sincrónico satura los núcleos físicos, mientras que el de Parameter Server crece y llega a igualarlo.

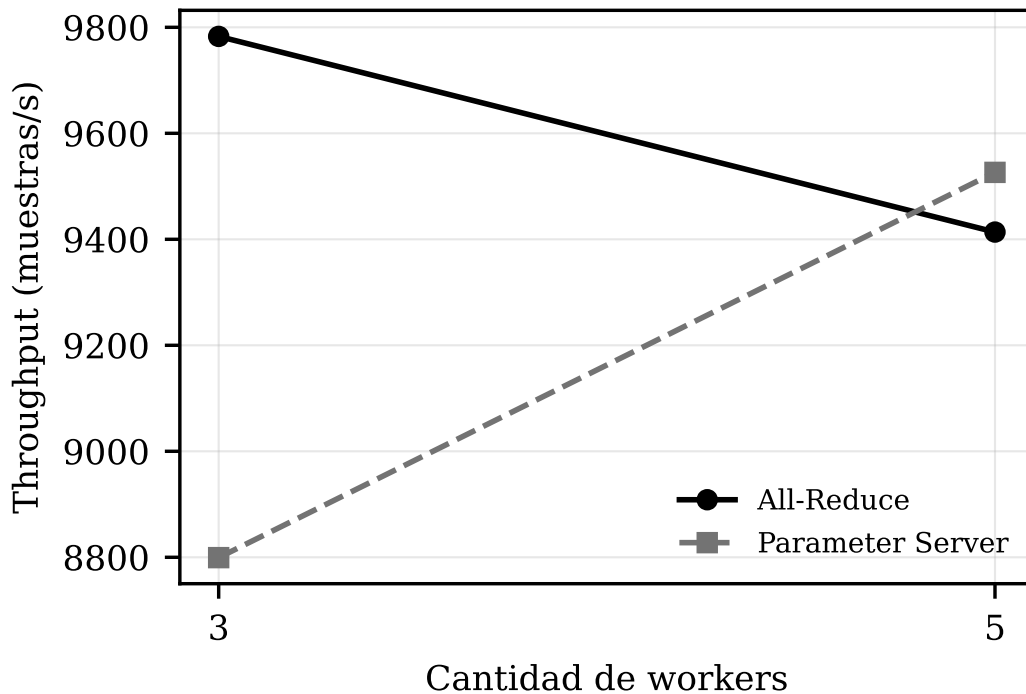


Figura 3: Throughput (muestras por segundo) frente al número de workers, con batch por worker fijo, en FashionMNIST. All-Reduce parte más alto pero Parameter Server lo alcanza al crecer los workers.

Parameter Server escala con pendiente positiva al sumar nodos, amortizando el costo fijo de sus servidores. Debe recordarse, sin embargo, que para hacerlo emplea dos nodos servidores adicionales que All-Reduce no necesita, y que la saturación de núcleos observada en All-Reduce es un artefacto del entorno de máquina única, no una propiedad del algoritmo.

11.3.6 Resultados: presión de comunicación

Para aislar el efecto del tamaño del gradiente se entrenaron las redes densas de tamaño creciente a 3 workers, con batch por worker fijo. El throughput se desploma al crecer el modelo: la comunicación, y no el cómputo, pasa a dominar. All-Reduce sostiene mayor throughput que Parameter Server en los tres tamaños, pero su ventaja se reduce al crecer el modelo, porque el fragmentado de parámetros entre los dos servidores reparte mejor la presión de comunicación.

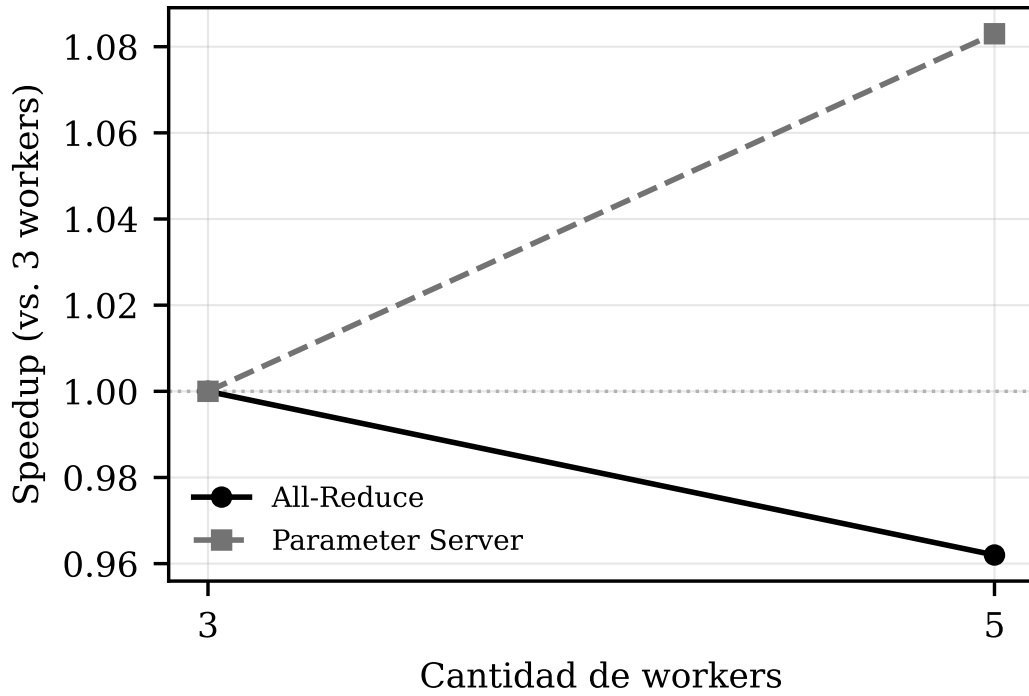


Figura 4: Speedup de throughput respecto de la configuración de 3 workers. Parameter Server escala con mayor pendiente que All-Reduce desde una base más baja.

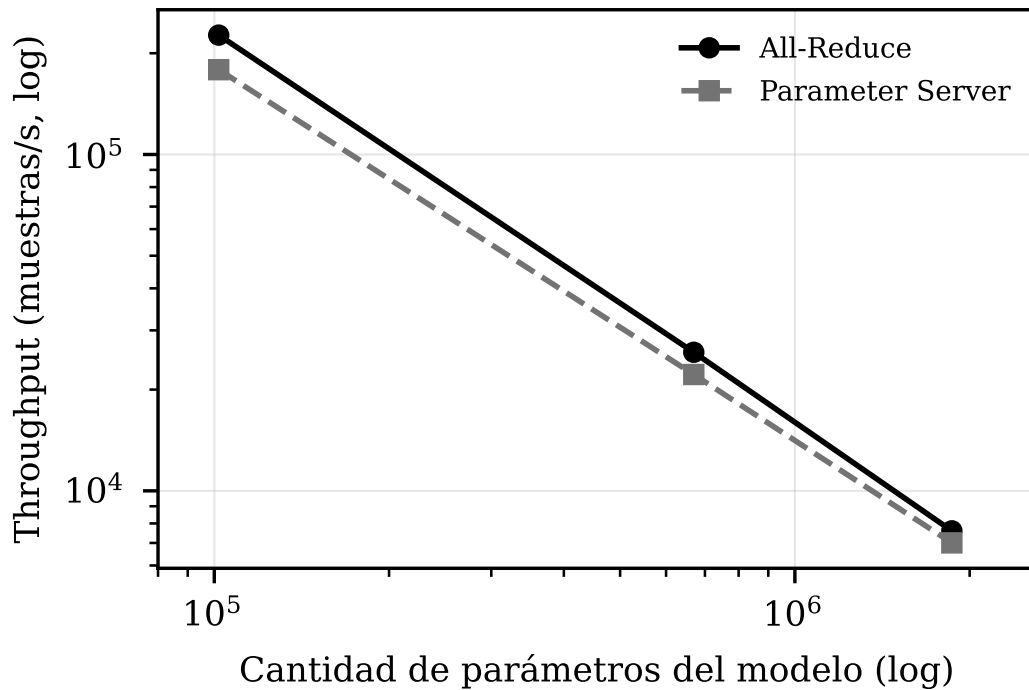


Figura 5: Presión de comunicación: throughput (escala logarítmica) frente al tamaño del modelo (escala logarítmica) a 3 workers. All-Reduce sostiene mayor throughput en los tres tamaños; la ventaja se reduce al crecer el modelo.

Este comportamiento se entiende mejor con una cuenta independiente de la red. En All-Reduce, cada nodo comunica aproximadamente $2\frac{W-1}{W}|g|$, esencialmente constante; en Parameter Server, el tráfico que ingresa a cada servidor crece de forma lineal con el número de workers, atenuado por el fragmentado entre S servidores. Esta métrica analítica explica tanto la eficiencia de ancho de banda de All-Reduce como el porqué el servidor se vuelve el punto de presión al sumar workers, y lo hace con independencia de la velocidad de la red subyacente, que es justamente la variable que el entorno de máquina única no permite explorar.

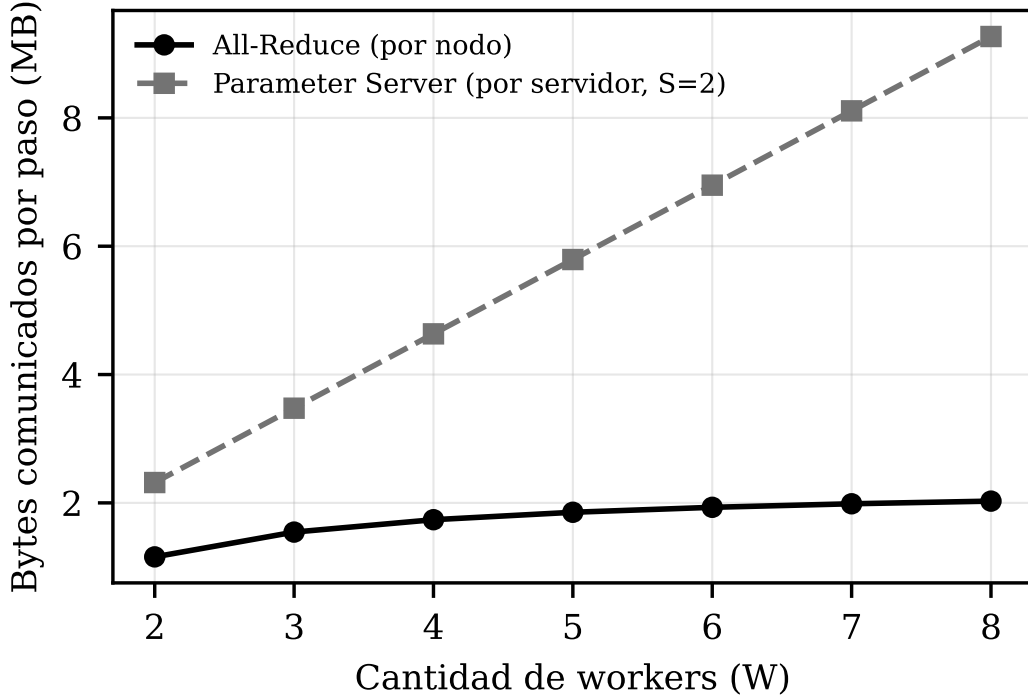


Figura 6: Volumen de datos comunicado por paso (métrica analítica, independiente de la red) frente al número de workers. En All-Reduce el volumen por nodo es casi constante; en Parameter Server el tráfico por servidor crece con los workers.

La compresión de gradientes, dispersa o cuantizada, es una mitigación conocida para esta presión (Aji & Heafield, 2017; Lin et al., 2018). O.N.O. la implementa, y de hecho el segundo estudio la utiliza; su evaluación aislada quedó fuera del alcance.

11.3.7 Discusión y matriz de decisión

El escenario evaluado, con clúster homogéneo, red rápida, entrenamiento sincrónico y gradientes densos, favorece a All-Reduce en todos los criterios donde la evidencia permite pronunciarse, y deja sin ejercitar los tres escenarios donde Parameter Server tendría ventaja estructural: heterogeneidad de nodos, tolerancia a rezagados y modelos que no entran en la memoria de una sola máquina. El modelo mayor evaluado también completó en All-Reduce, de modo que ni siquiera ese último llegó a ponerse a prueba. La matriz resume criterio por criterio.

Cuadro 8: Matriz de decisión cualitativa All-Reduce frente a Parameter Server, acotada al entorno evaluado.

Criterio	All-Reduce	Parameter Server
Convergencia	Empate: misma semántica síncrona.	Empate: misma semántica síncrona.
Throughput / tiempo hasta exactitud	Favorable: sin servidor central.	Menor: sobrecarga del servidor.
Escalado al sumar nodos	Escala bien, pendiente base.	Amortiza su costo fijo: pendiente algo mayor.
Eficiencia de hardware	No requiere nodos servidores extra.	Requiere nodos servidores dedicados.
Presión de comunicación / modelos grandes	Sostiene la ventaja.	El fragmentado acorta la brecha.
Heterogeneidad / rezagados / asincronía	Síncrono: sensible a rezagados.	Ventaja estructural; no ejercitada aquí.

11.3.8 Limitaciones y conclusión del estudio

El entorno simulado atenúa el costo de comunicación, y esa atenuación beneficia a Parameter Server más de lo que lo haría una red real: la ventaja medida de All-Reduce es, por lo tanto, un piso y no un techo. El estudio se restringe además al régimen sincrónico y a datos distribuidos de forma homogénea. O.N.O. es un sistema académico y no compite con frameworks industriales como Horovod (Sergeev & Del Balso, 2018) o BytePS (Jiang et al., 2020), que se usan aquí solo como marco conceptual.

La conclusión, entonces, es condicional y no absoluta: bajo estas condiciones conviene All-Reduce, y determinar cuál conviene fuera de ellas requiere ejercitar los escenarios que quedaron sin cubrir.

11.4 Estudio II: impacto de las épocas offline

11.4.1 Pregunta e hipótesis

Permitir que los nodos operen de forma autónoma durante varias épocas posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia entre los pesos de los nodos, un fenómeno conocido como *client drift* (McMahan et al., 2017). O.N.O. expone este mecanismo como un parámetro de configuración, las épocas offline (E), lo que permite estudiarlo de forma directa.

La pregunta es: ¿cómo impactan las épocas offline en la velocidad y en la eficacia del modelo entrenado? La hipótesis de partida era la que motiva la técnica: que un E mayor reduciría el tiempo total a costa de alguna pérdida de exactitud, y que existiría un punto de equilibrio aprovechable.

11.4.2 Metodología

Se ejecutaron entrenamientos sobre una misma arquitectura, conjunto de datos e hiperparámetros, variando únicamente el algoritmo distribuido, la cantidad de nodos y el valor de E . Todos los ensayos

se realizaron de forma secuencial sobre la misma máquina, sumando un total de 10,5 horas de ejecución.

Cuadro 9: Configuración de los ensayos. En Parameter Server, la mitad de los nodos son workers y la otra mitad servidores.

Categoría	Parámetro	Valores
Modelo y datos	Dataset	MNIST completo (LeCun et al., 1998)
	Arquitectura	LeNet-5
	Función de pérdida	Entropía cruzada
Optimización	Optimizador	Adam ($\alpha = 0,01$; $\beta_1 = 0,9$; $\beta_2 = 0,999$; $\epsilon = 10^{-8}$)
	Tamaño de mini-batch	64
	Épocas máximas	48
Infraestructura	Serialización	Dispersa ($r = 0,5$)
Variables libres	Algoritmo	All-Reduce, Parameter Server sincrónico
	Nodos	2, 4, 6, 8, 10
	Épocas offline	0, 1, 2, 3, 4

Al parametrizar $E > 0$, los nodos entrenan aislados durante múltiples ciclos y el tráfico de red se reduce de forma aproximadamente inversamente proporcional a E . Al alcanzar la etapa de sincronización, la consolidación se realiza por promedio directo de los gradientes parciales, de modo que bajo descenso por gradiente la actualización responde a

$$W_{t+1} = W_t - \frac{\lambda}{N} \sum_{i=1}^N G_i.$$

Los ensayos se realizaron con la serialización dispersa que implementa el sistema, siguiendo el enfoque de Aji y Heafield (Aji & Heafield, 2017). El parámetro r acota el muestreo del gradiente que se transmite: se calcula la cardinalidad efectiva del mensaje como

$$k = \text{round}(|g| \cdot (1 - r)),$$

y ese valor k se utiliza para escoger los k elementos de mayor magnitud absoluta del tensor de gradientes local, fijando un umbral mínimo. Los elementos descartados se acumulan en un gradiente residual que se considera en el mensaje siguiente. Con $r = 0,5$, la mitad de los componentes queda fuera de cada mensaje.

El entorno de ejecución fue un procesador Intel Core i5-8350U (4 núcleos físicos, 8 hilos lógicos, 1,7 GHz base y 3,6 GHz turbo) con 8 GB de memoria DDR4, sobre Ubuntu 24.04 LTS, Docker 29.6.1, Rust 1.96.1 y CPython 3.14.0.

11.4.3 Resultados: tiempo de ejecución

La variación de E no indujo mejoras significativas en los tiempos de ejecución: las curvas de cada configuración se solapan de forma estricta en todos los ensayos. Dado que el sistema se ejecuta en una única máquina, la supresión del costo de comunicación no alcanza a compensar la sobrecarga de cómputo, contrariamente a la expectativa inicial.

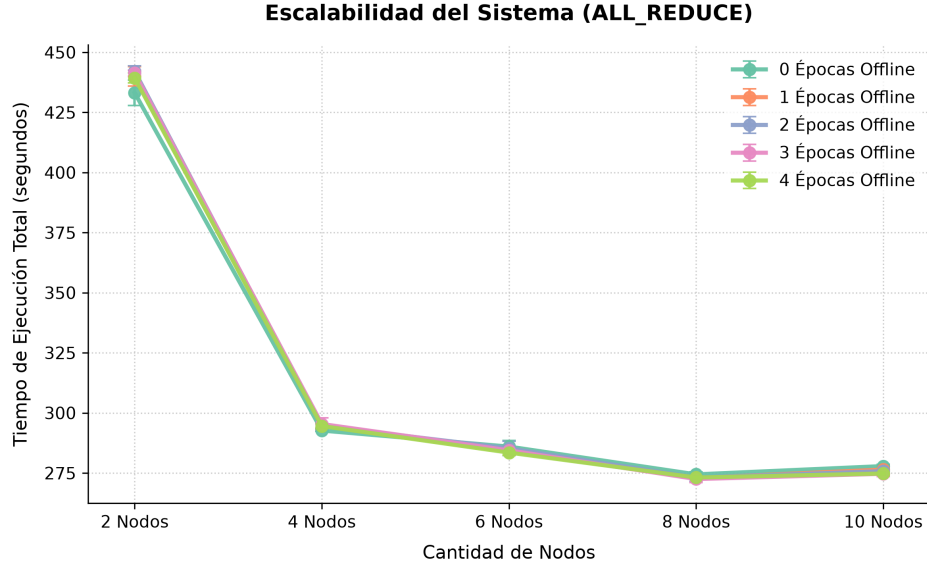


Figura 7: Comparación de tiempos de ejecución bajo distintas cantidades de épocas offline con All-Reduce.

La escalabilidad del rendimiento se estanca al superar los 4 nodos, y en All-Reduce la elevación a 10 nodos degrada el desempeño global por la alta contención de CPU.

En Parameter Server, en cambio, la configuración de 10 nodos (5 workers y 5 servidores) reduce el tiempo respecto de configuraciones menores. La explicación es contraintuitiva pero consistente con el entorno: mientras que en All-Reduce los procesos operan en cómputo continuo, el esquema sincrónico de Parameter Server introduce estados de inactividad obligatorios mediante barreras, y esa inactividad disminuye la contención, permitiendo una alternancia más eficiente de los hilos de ejecución sobre los cuatro núcleos físicos disponibles. All-Reduce resulta, no obstante, intrínsecamente más veloz en configuraciones de pocos nodos, por su distribución balanceada de responsabilidades sin dependencias centralizadas.

11.4.4 Resultados: exactitud

El impacto de E se manifiesta con mayor claridad en la exactitud, por ser una métrica independiente de las limitaciones del hardware de ejecución. En ambos algoritmos, la exactitud final decrece de forma monótona al incrementar E . La introducción de una única época offline ($E = 1$), que reduce la frecuencia de sincronización global a la mitad, ya provoca un deterioro inmediato de aproximadamente un punto porcentual.

La degradación se acentúa de forma proporcional al número de nodos. Al incrementar las entidades distribuidas, las particiones locales del conjunto de datos se reducen y se vuelven potencialmente

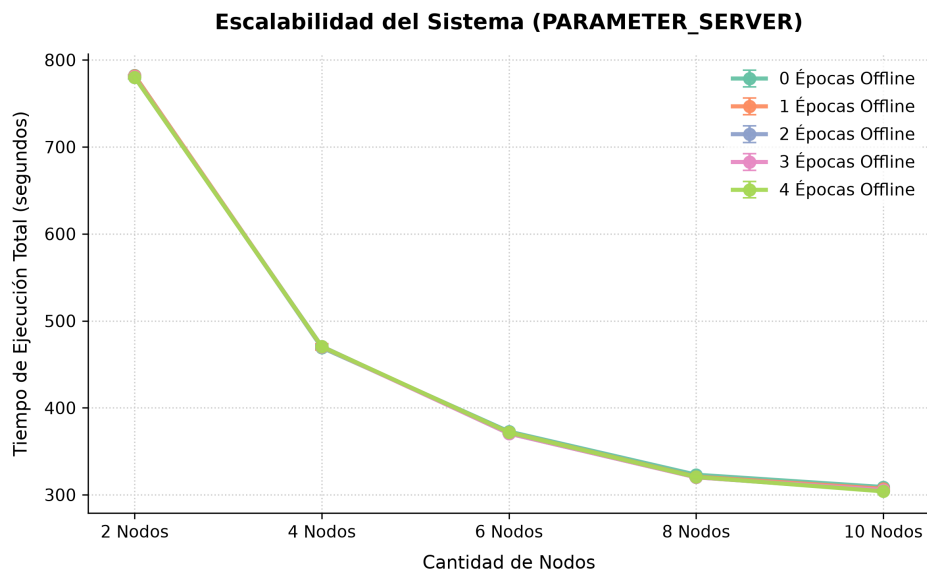


Figura 8: Comparación de tiempos de ejecución bajo distintas cantidades de épocas offline con Parameter Server.

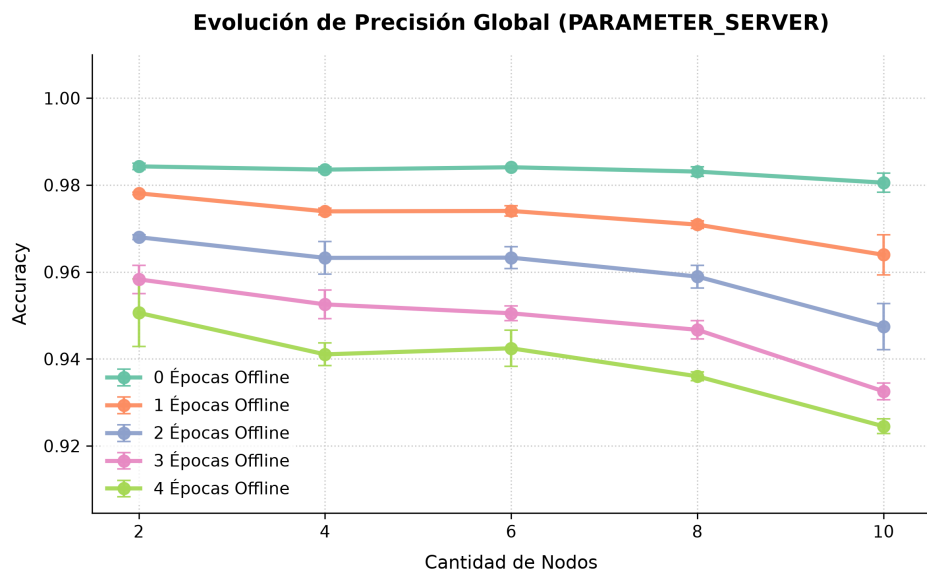


Figura 9: Comparación de exactitud bajo distintas cantidades de épocas offline con Parameter Server.

sesgadas; la falta de sincronización frecuente exacerba el impacto de esos sesgos locales, perjudicando la convergencia global.

En All-Reduce la pérdida de exactitud es más severa que en Parameter Server. Esto se explica por el diseño experimental: bajo las condiciones evaluadas, All-Reduce duplica el número de workers activos al prescindir de servidores dedicados, lo que fragmenta aún más el conjunto de datos e incrementa el aislamiento operativo. La integración tardía de parámetros fuerza entonces correcciones abruptas del gradiente global.

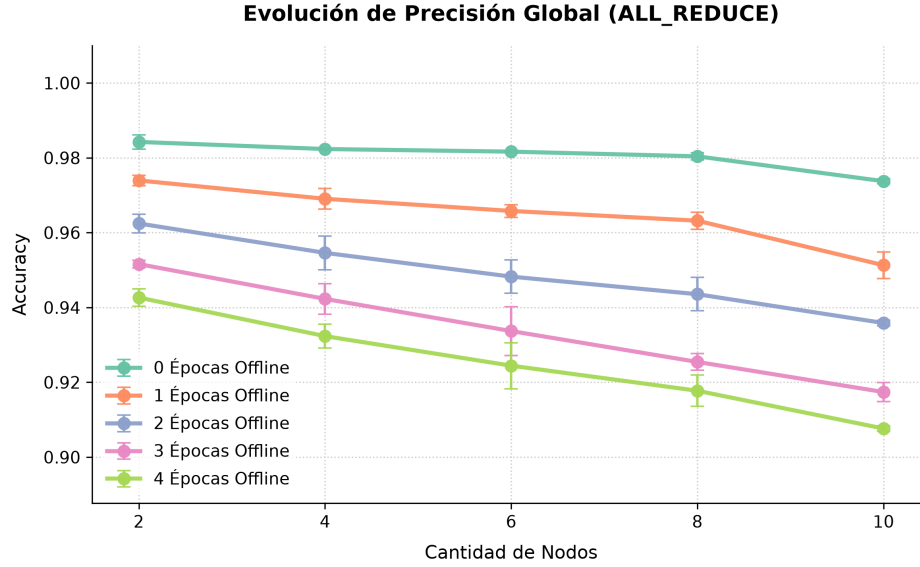


Figura 10: Comparación de exactitud bajo distintas cantidades de épocas offline con All-Reduce.

11.4.5 Resultados: inestabilidad de la pérdida

A partir del historial de entrenamiento se seleccionaron trayectorias representativas que confirman que el incremento de E introduce inestabilidad en la función de pérdida, evidenciada mediante oscilaciones y picos proporcionales al valor de E .

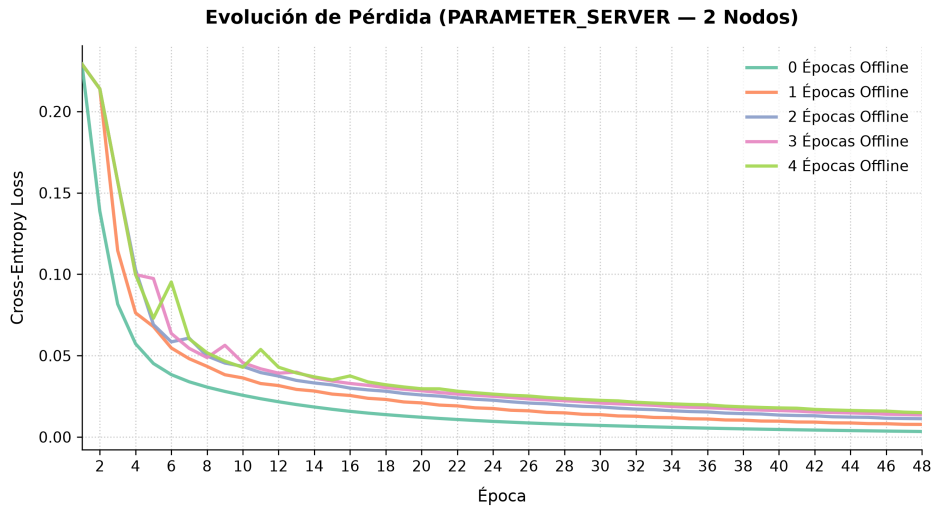


Figura 11: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en Parameter Server (2 nodos).

La dinámica del optimizador presenta mayores dificultades en las épocas iniciales, la fase asociada a las correcciones de mayor magnitud, y luego se estabiliza al aproximarse a un mínimo local. Sin embargo, los valores de convergencia residual son sistemáticamente más altos a mayor E . El fenómeno sugiere que el desacoplamiento prolongado restringe la capacidad del optimizador para alcanzar el

mínimo, induciendo un comportamiento análogo al de una tasa de aprendizaje excesivamente alta.

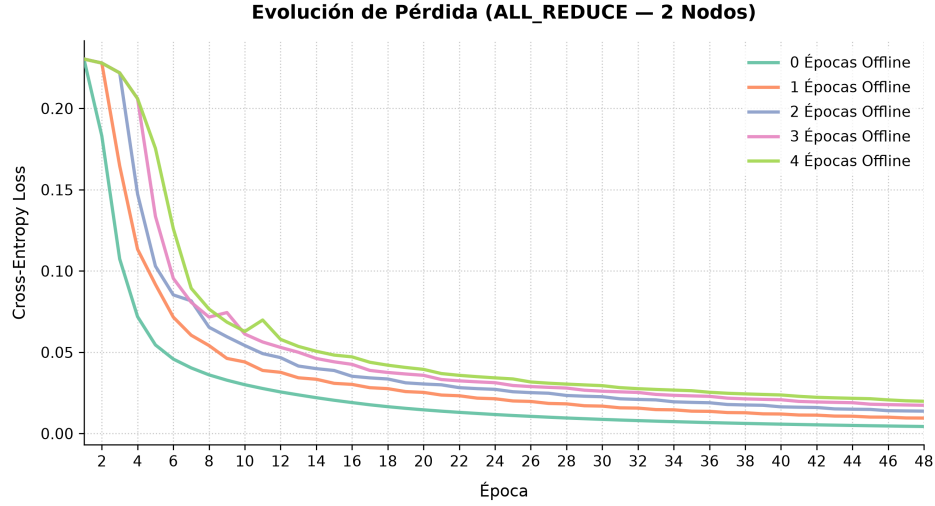


Figura 12: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en All-Reduce (2 nodos).

En All-Reduce se observa un patrón similar, aunque con oscilaciones relativamente más atenuadas, lo que revierte la hipótesis inicial que preveía mayor inestabilidad en el esquema descentralizado.

11.4.6 Discusión y limitaciones

La ventaja de introducir épocas offline es estrictamente reducir el tiempo de entrenamiento. En un entorno local simulado, eliminar el costo de comunicación no compensa el costo de obtener un peor modelo: en estos experimentos no se evidenció ninguna ventaja de utilizar un E mayor a 0.

En contraposición, en un despliegue multimáquina sobre una red física, retrasar la agregación de gradientes puede resultar crítico para amortizar el tiempo de comunicación. La decisión depende de tres factores: el tamaño del modelo, la velocidad de la red y la cantidad de nodos. Con modelos más grandes, los mensajes que contienen gradientes y parámetros son también más grandes. Para entrenamientos con mucho dato y poco modelo, el cuello de botella es el cómputo de gradientes; en cambio, para entrenamientos con poco dato y mucho modelo, donde la comunicación toma protagonismo, incrementar E puede ser útil para reducir los tiempos de ejecución.

Un resultado del estudio que merece señalarse es que la desestabilización afecta negativamente al modelo de forma predecible, lo que abre una línea concreta: evaluar estrategias de activación dinámica de las épocas offline, introduciéndolas exclusivamente en fases avanzadas del entrenamiento, cuando la optimización principal ha concluido, de modo que la exploración independiente de un worker en torno a un mínimo local pueda guiar favorablemente al resto tras la sincronización.

11.4.7 Conclusión del estudio

El uso de épocas offline en O.N.O. altera el balance entre comunicación y fidelidad algorítmica. En la infraestructura simulada, incrementar E perjudicó al modelo entrenado de manera predecible pero sin aportar ganancias de velocidad, debido a las limitaciones del hardware empleado. Queda

planteado validar el sistema bajo configuraciones multimáquina reales, comparando arquitecturas de modelos de distintos tamaños al variar E .

11.5 Estudio III: Strategy Switch

Sección pendiente de redacción. El tercer estudio caracteriza *Strategy-Switch* (Provatas et al., 2025): en qué medida iniciar el entrenamiento en régimen sincrónico de All-Reduce y promover trabajadores a servidores de parámetros una vez que los gradientes se estabilizan permite combinar la exactitud del primer régimen con la reducción de tiempo del segundo. Sigue la estructura de los dos estudios anteriores, de modo que los tres resulten comparables.

11.6 Síntesis de la validación

Los tres estudios responden preguntas distintas y convergen en la misma observación sobre el entorno: la simulación del clúster sobre una única máquina gobierna todos los resultados de rendimiento. Los resultados de convergencia y exactitud, en cambio, son independientes de esa limitación y pueden leerse con mayor confianza.

La segunda observación transversal es que el sistema cumplió su propósito. Validada la corrección de ambas estrategias por la vía de su convergencia, las diferencias de tiempo, throughput y escalabilidad admiten atribuirse al algoritmo y no a su implementación, que era exactamente el problema que este trabajo se propuso resolver.

12 Cronograma de las actividades realizadas

El plan de actividades de la propuesta fijó dieciséis tareas repartidas en cuatro hitos sobre dos cuatrimestres, con una carga estimada de 1500 horas de equipo. Esta sección contrasta ese plan con la ejecución real, que se extendió entre agosto de 2025 y julio de 2026.

12.1 Fases efectivamente recorridas

El desarrollo puede reconstruirse en once fases. Las fechas son aproximadas y corresponden a rangos de actividad, ya que varias fases se solaparon.

Cuadro 10: Fases del desarrollo efectivamente recorridas.

Fase	Período	Contenido
0	Ago–Nov 2025	Propuesta, relevamiento bibliográfico y un primer diseño descartado
1	Nov 2025–Ene 2026	Capa de comunicación
2	Dic 2025–Feb 2026	Parameter Server y Worker
3	Ene–Mar 2026	Motor de redes neuronales
4	Feb–Mar 2026	Orchestrator, interfaz de terminal e interfaz de Python
5	Mar 2026	Multi-servidor, conjuntos de datos y eficiencia de red
6	Mar–May 2026	All-Reduce en anillo y unificación del nodo
7	Abr–May 2026	Early stopping y la saga de bloqueos
8	May–Jun 2026	Strategy Switch, topología y benchmarks
9	Jun 2026	Endurecimiento y exposición de funcionalidad
10	Jun–Jul 2026	Consolidación del benchmark y auditoría de corrección
11	Jul 2026	Retracción de la tolerancia a fallos, optimización final y documentación

12.2 Contraste entre lo planificado y lo ejecutado

Lo que se cumplió según lo previsto. El núcleo del plan se completó: el sistema base parametrizable en Rust con su biblioteca de redes neuronales propia (tarea 4), los tres algoritmos (tareas 5 a 7), las optimizaciones de comunicación y sincronización (tareas 8 y 9), las dos interfaces (tarea 10), las pruebas (tarea 11), la simulación sobre distintas configuraciones de nodos (tarea 12) y el análisis de resultados con sus gráficos (tarea 14).

Lo que llevó más tiempo del estimado. Tres bloques se desviaron de forma significativa. El primero es la fase 0: casi cinco meses dedicados a documentación, relevamiento y trámites antes de escribir código productivo, con un primer diseño de arquitectura descartado por completo. El segundo es el motor de redes neuronales (tarea 4), cuya derivación manual de la propagación hacia atrás y la implementación de las capas convolucionales insumieron alrededor de dos meses de depuración de gradientes, muy por encima de lo previsto para un componente que la propuesta trataba como parte de una tarea mayor. El tercero es la saga de bloqueos de coordinación desatada por el early stopping (fase 7), que no figuraba en el plan bajo ninguna tarea y consumió un mes completo.

Lo que no se ejecutó. La tarea 13, optimización de carga de cómputo en configuraciones heterogéneas, con 150 horas estimadas, no se abordó: dependía de disponer de máquinas físicas con capacidades distintas, algo que el entorno de simulación no permite representar de forma convincente.

Lo que se ejecutó sin estar planificado. Aparecieron cuatro líneas no previstas: la selección de topología mediante estadísticas de latencia, la auditoría de corrección distribuida que surgió de la construcción de la suite de benchmarks, la optimización del motor de convolución, y las dos monografías que constituyen los estudios experimentales de este informe, que excedieron en profundidad lo que la tarea 14 contemplaba.

Una observación sobre la estimación. La distribución de horas de la propuesta asignaba 200 horas a investigar las implementaciones distribuidas de TensorFlow y PyTorch (tareas 2 y 3). En la práctica ese estudio se realizó de forma mucho más acotada y en modalidad de consulta puntual, y el esfuerzo se redistribuyó hacia la implementación y la depuración del motor de aprendizaje. La lección de estimación es la habitual y no por conocida menos real: el esfuerzo se subestima sistemáticamente en aquello que no se conoce de antemano, que en este trabajo fue todo lo relativo a la corrección numérica del entrenamiento y a la coordinación distribuida.

12.3 Carga horaria efectiva

Cuadro 11: Contraste entre la carga horaria estimada en la propuesta y la efectivamente dedicada.

Nro.	Tarea	Horas estimadas	Horas reales
1	Leer y analizar los trabajos previos	100	<i>[a completar]</i>
2	Investigar la implementación distribuida de TensorFlow	50	<i>[a completar]</i>
3	Investigar la implementación distribuida de PyTorch	50	<i>[a completar]</i>
4	Desarrollar el sistema distribuido en Rust	150	<i>[a completar]</i>
5	Implementar Parameter Server	100	<i>[a completar]</i>

Nro.	Tarea	Horas estimadas	Horas reales
6	Implementar All-Reduce	100	<i>[a completar]</i>
7	Implementar Strategy-Switch	100	<i>[a completar]</i>
8	Optimizaciones de comunicación entre nodos	150	<i>[a completar]</i>
9	Optimizaciones de sincronización de las copias del modelo	150	<i>[a completar]</i>
10	Interfaz de funciones externas en Python e interfaz de terminal	50	<i>[a completar]</i>
11	Pruebas unitarias y de integración	50	<i>[a completar]</i>
12	Simulación sobre distintas configuraciones de nodos	100	<i>[a completar]</i>
13	Optimizaciones de carga en configuraciones heterogéneas	150	0 (no ejecutada)
14	Análisis de resultados y gráficos	100	<i>[a completar]</i>
15	Documentación del código y del proceso	50	<i>[a completar]</i>
16	Informe final	50	<i>[a completar]</i>
	Total	1500	<i>[a completar]</i>

13 Riesgos materializados y lecciones aprendidas

La propuesta identificó cuatro riesgos iniciales. Tres se materializaron, uno de ellos con una forma distinta de la prevista, y aparecieron otros que no estaban contemplados. Esta sección los recorre con la evidencia del historial del proyecto y cierra con las lecciones que dejaron.

13.1 Riesgos previstos que se materializaron

La complejidad de la implementación distribuida en Rust. Fue el riesgo dominante. Las garantías de *fearless concurrency* del lenguaje cumplieron lo que prometen, ya que ninguna condición de carrera sobre memoria compartida llegó a la rama principal, pero desplazaron la dificultad hacia otro terreno: el de la coordinación distribuida, que el compilador no puede verificar porque no es un problema de memoria sino de protocolo. La dificultad real fue mantener sincronizado el estado de fase de cada entidad dentro del algoritmo que está ejecutando, especialmente en All-Reduce, donde cada nodo avanza por pasos que dependen del avance de sus vecinos.

De ahí salieron los dos bloqueos más costosos del trabajo. El primero es de membresía: el contador de la barrera se fijaba en N trabajadores al inicio, y cuando uno detenía su entrenamiento y se desconectaba, la barrera quedaba esperando indefinidamente a alguien que ya se había ido. El segundo es de control de flujo: el anillo de All-Reduce se trababa con modelos grandes porque enviaba y recibía de forma secuencial, de modo que todos los nodos intentaban enviar antes de recibir y el buffer del socket se llenaba sin que nadie lo drenara. No producía error ni consumo de CPU, simplemente se detenía.

La disponibilidad de hardware para simulaciones representativas. Se materializó, y es la limitación transversal del trabajo. La mitigación prevista en la propuesta, contenedores con límites de recursos, no alcanza a suplir lo que no está. La respuesta que se dio, y que se considera la decisión metodológica más valiosa del trabajo, fue instrumentar la comparación de modo que el lector pueda separar lo que es propiedad del algoritmo de lo que es artefacto del entorno; el mecanismo y su alcance se describen en *Experimentación y validación*.

La amplitud del alcance y la estimación de tiempos. La mitigación prevista, priorizar un sistema base funcional y abordar las optimizaciones de forma incremental, se aplicó y funcionó. El costo fue que algunas líneas postergadas no llegaron a cerrarse, según se detalla más abajo.

La dependencia de trabajos previos. Se materializó de una forma que no se había anticipado. Lo previsto era que la interpretación de un algoritmo publicado pudiera diferir de lo documentado; lo que ocurrió fue más sutil: los supuestos de los papers no siempre se cumplen en el régimen propio.

13.2 Riesgos no previstos

Aplicar un algoritmo sin verificar la validez de sus supuestos. El sistema implementa la actualización de parámetros libre de bloqueos siguiendo el trabajo de Niu et al. (Niu et al., 2011), detrás de un *trait* que permite elegirla en tiempo de ejecución. Una auditoría tardía mostró dos problemas independientes.

El primero es de corrección: la implementación obtiene dos referencias mutables a la misma memoria desde múltiples hilos, lo cual en Rust es comportamiento indefinido. No es equivalente a la condición

de carrera “benigna” del trabajo original, escrito en C++, donde una carrera sobre memoria plana queda definida por la implementación; en Rust el compilador asume que no ocurre y optimiza en consecuencia. El síntoma no fue un fallo sino un cuelgue silencioso que solo se manifestaba con varios trabajadores concurrentes, razón por la cual las pruebas existentes, que ejercitaban únicamente la variante con bloqueo, nunca lo detectaron.

El segundo es conceptual. *Hogwild!* converge bien con gradientes malos, porque su argumento se apoya en que las colisiones entre actualizaciones concurrentes son infrecuentes. Una red convolucional densa, que es lo que O.N.O. entrena, produce gradientes en los que cada parámetro se actualiza en cada paso y las colisiones son masivas. El supuesto del paper no se cumple en el régimen propio.

El análisis dejó al descubierto una tensión de diseño irreducible: no se pueden tener a la vez un optimizador genérico con estado, una implementación realmente libre de bloqueos y corrección. Usar operaciones atómicas daría una implementación correcta pero solo con un optimizador sin estado, ya que el ciclo de lectura-modificación-escritura del estado interno de Adam o de momentum no admite ser libre de bloqueos. Poner un bloqueo por fragmento elimina el problema, pero entonces deja de ser la técnica del paper. La variante quedó fuera de la suite de benchmarks y las dos alternativas se detallan en *Trabajos futuros*.

Una decisión de arquitectura temprana condiciona una funcionalidad tardía. Es el riesgo más instructivo del proyecto, porque no se trata de un error sino de un compromiso que solo se hizo visible dos fases después de haberlo asumido. Representar el modelo como un único buffer plano, con vistas sin estado sobre desplazamientos y formas, fue la decisión correcta y sostuvo el rendimiento del sistema durante todo el trabajo. Su consecuencia es que el tipo de mensaje del protocolo toma una referencia a los bytes del buffer en lugar de poseerlos, y un mensaje que referencia memoria ajena no puede moverse libremente entre tareas asíncronas. Meses después, esa restricción resultó ser exactamente la que bloqueaba la capa de reconexión sobre la que se apoyaría la recuperación ante caídas de nodos.

La lección no es que la decisión original haya sido mala, sino que las decisiones de arquitectura tienen un alcance temporal mayor que el de la funcionalidad para la que se toman, y el costo de revisarlas crece con la distancia: cuando el compromiso se hizo visible, revertirlo implicaba tocar cinco crates y 41 archivos. A eso se suma un error de proceso propio: el análisis de alternativas que finalmente identificó el obstáculo llegó después de dos intentos de implementación, y no antes. El diseño debió preceder al código.

13.3 Desvíos de alcance

La tarea de optimización de carga en configuraciones heterogéneas no se abordó. Dependía de disponer de máquinas físicas con capacidades distintas, algo que el entorno de simulación no permite representar de forma convincente.

Las pruebas de integración de alto nivel fueron reemplazadas. Aquellas que levantaban un clúster completo desde Python se abandonaron en favor de la suite de benchmarks: sin un estándar acordado para las configuraciones de Python y Docker que requerían, su mantenimiento superaba la confianza que aportaban, y la suite cubrió el mismo terreno con un propósito adicional.

Un conjunto de líneas se cerró deliberadamente sin implementar. En una sesión de

definición de alcance previa al cierre se descartaron de forma explícita el multiplexado de sesiones, la estandarización del orden de bytes, el consumo de conjuntos de datos por flujo y la parametrización genérica del tipo de punto flotante. Se registran como descartes conscientes y no como pendientes.

13.4 Lecciones aprendidas

1. La instrumentación es una herramienta de detección de errores, no un anexo de medición. Es la lección central del trabajo. La suite de benchmarks se construyó para comparar estrategias y terminó destapando tres errores de corrección del sistema distribuido que llevaban meses en el código: que el anillo de All-Reduce sumaba los gradientes en lugar de promediarlos, de modo que la tasa de aprendizaje efectiva escalaba con la cantidad de trabajadores; que los servidores de parámetros almacenaban las capas ordenadas por tamaño en lugar de por su orden en el modelo; y que al conmutar de estrategia los fragmentos de parámetros se asignaban siguiendo un orden distinto del que usaba el trabajador para indexarlos. Los tres eran silenciosos: no provocaban fallos, solo degradaban la exactitud, y ninguna prueba unitaria los habría encontrado porque cada componente hacía correctamente lo que le correspondía.

Vale detenerse en por qué el error del orden de capas sobrevivió tanto tiempo: todas las redes de prueba tenían capas monótonamente decrecientes (784 a 128 a 64 a 10), de modo que ordenar por tamaño coincidía por casualidad con el orden del modelo. Un conjunto de pruebas que no varía la forma de sus datos no puede distinguir una implementación correcta de una que acierta por accidente.

2. El paralelismo no siempre es la respuesta. La optimización del motor de aprendizaje se abordó con el criterio declarado de optimizar primero en un solo hilo y recién después ir por varios. Se probaron tres enfoques para acelerar la convolución. El primero fue calcularla en el dominio de la frecuencia: como la convolución allí es un producto punto a punto, transformar entrada y filtro, multiplicar y antitransformar cambia el costo por el de la transformada. Se descartó porque agregaba más sobrecarga que optimización, dado que los problemas que resuelve este trabajo no computan convoluciones con matrices lo suficientemente grandes como para amortizar el costo de la transformada. El segundo fue la paralelización con hilos, que se implementó y también se descartó, con la misma conclusión empírica: más sobrecarga que beneficio para el tamaño de problema evaluado. Lo que finalmente funcionó fue reformular la convolución como una multiplicación de matrices, para explotar las rutinas de álgebra lineal optimizadas y la localidad de caché, con una mejora de 2,27 veces en el tiempo total. La intuición inicial de agotar la optimización de un solo hilo antes de paralelizar resultó correcta.

3. La duplicación de documentación produce divergencia igual que la del código. El esquema de configuración estaba replicado en tres archivos distintos y los tres habían divergido, describiendo nombres de campos que ya no existían. Se resolvió creando un documento canónico y eliminando las copias. Es la misma regla de única fuente de verdad que el proyecto había tenido que aplicar en el código, donde llegaron a convivir dos funciones de particionado distintas que solo coincidían en el caso de un único servidor.

4. Cerrar y reabrir una rama produce buenas revisiones y mala trazabilidad. El equipo adoptó de forma sistemática el patrón de cerrar ramas de trabajo en curso y abrir una nueva y limpia en lugar de iterar sobre la existente, y ocurrió al menos seis veces en componentes centrales.

Como práctica produce un historial de cambios legible y revisiones acotadas; su costo es que el razonamiento queda repartido entre pull requests cerradas que un lector externo no encuentra si no las busca, y buena parte del esfuerzo de reconstrucción del proceso para este informe se debió a eso.

14 Impactos económico, social y ambiental del proyecto

La propuesta incluyó una evaluación preliminar de impacto, formulada con los conocimientos disponibles al inicio del trabajo. Se revisa ahora a la luz de lo efectivamente construido y medido, señalando tanto lo que se sostiene como lo que debe matizarse.

Impacto económico. El entrenamiento de modelos de aprendizaje profundo concentra buena parte de su costo en el tiempo de cómputo y en la infraestructura necesaria para sostenerlo. La premisa de que distribuir el entrenamiento reduce ese tiempo se verificó, pero los experimentos obligan a acotarla: la ganancia no es automática y depende de la relación entre el costo de cómputo y el costo de comunicación. Se observó que, al aumentar la cantidad de nodos por encima de la capacidad física disponible, el rendimiento se estanca e incluso se degrada; y que el volumen de comunicación crece con el tamaño del modelo hasta dominar el tiempo total. La conclusión económica honesta es que la distribución reduce el costo cuando la carga de cómputo por nodo justifica el costo de coordinarlos, y que un sistema como el desarrollado sirve justamente para determinar dónde está ese umbral antes de invertir en infraestructura.

Un segundo efecto económico, más directo, es que el trabajo se desarrolló íntegramente sobre hardware propio de los integrantes y con herramientas de código abierto, sin licencias ni servicios pagos. El costo del proyecto se reduce, por tanto, a las horas-persona invertidas. Esa misma propiedad se traslada al producto: O.N.O. no requiere aceleradores especializados ni infraestructura de nube para ser utilizado o extendido.

Impacto social. El trabajo se publica como base abierta y parametrizable, con el objetivo de acercar el entrenamiento distribuido a equipos que no cuentan con grandes clústeres y de ofrecer un punto de partida para la investigación y la docencia en el área. La disponibilidad de implementaciones de referencia de los tres algoritmos fundamentales sobre una misma infraestructura, junto con una interfaz accesible desde Python y una interfaz de terminal que permite observar el entrenamiento en vivo, facilita que otros retomen, comparen y extiendan el trabajo. El valor pedagógico resultó más concreto de lo previsto: al no delegar nada en un framework existente, el sistema expone la mecánica completa del entrenamiento distribuido, desde la propagación hacia atrás hasta el protocolo de red, y en ese sentido funciona como material de estudio además de como herramienta.

El reverso. Toda mejora en la eficiencia del entrenamiento de modelos de aprendizaje profundo es una tecnología de doble uso: reduce la barrera de entrada tanto para aplicaciones beneficiosas como para aplicaciones que no lo son. Este trabajo no incorpora mecanismos de mitigación al respecto, y su escala es la de un sistema académico, pero la observación forma parte de una evaluación honesta.

Impacto ambiental. El tiempo de cómputo se traduce de forma directa en consumo energético, de modo que cualquier reducción del tiempo total de entrenamiento tiene un correlato en el consumo. Este es el punto en el que la evaluación preliminar requiere la corrección más fuerte: la distribución introduce un costo adicional de comunicación y replica el cómputo de coordinación en cada nodo, de modo que un entrenamiento distribuido sobre N nodos consume más energía agregada que el mismo entrenamiento secuencial, aun cuando termine antes.

La distribución se justifica ambientalmente, entonces, cuando permite entrenar modelos que de otro modo no serían viables, o cuando el paralelismo se despliega sobre máquinas físicas independientes que de todas formas estarían encendidas. Las técnicas de reducción de tráfico implementadas en el sistema, la codificación de gradientes en media precisión y el protocolo de gradientes dispersos,

apuntan precisamente a que la ejecución distribuida traduzca su ganancia de tiempo en un uso más eficiente de los recursos y no solo en un mayor consumo agregado. Cuantificar ese ahorro energético de forma directa excedió el alcance del trabajo y queda planteado como línea futura.

15 Trabajos futuros

El sistema quedó en un estado que admite ser retomado, y varias de las líneas que siguen tienen su diseño ya especificado en el repositorio. Se ordenan por relevancia.

Tolerancia a fallos. Es la frontera abierta más importante y la que el trabajo intentó cerrar sin lograrlo. El comportamiento esperado ya está especificado por entidad: el orquestador debe encolar los mensajes destinados a un par caído hasta que este se reconecte; el servidor de parámetros debe distinguir un trabajador lento de uno muerto; y el trabajador de All-Reduce debe poder esperar la reconexión de sus vecinos en el anillo. Lo que falta es la pieza habilitante, una capa de reconexión en el transporte, y retomarla exige resolver primero el obstáculo documentado en las lecciones aprendidas: el tipo de mensaje del protocolo referencia los bytes del buffer del modelo en lugar de poseerlos, y por eso no puede moverse entre tareas asincrónicas. Cualquier intento futuro debería empezar por decidir ese punto de diseño antes de escribir código: o el mensaje pasa a poseer sus datos, o se busca un mecanismo que evite tener que moverlo.

Un requisito adicional ya relevado: dado que el sistema admite varias entidades sobre la misma dirección IP con puertos distintos, la reconexión necesita un protocolo por el cual cada extremo anuncie, al establecerse la conexión, el puerto en el que escuchará si se cae y revive.

Validación sobre un clúster físico real. Es la línea de mayor impacto sobre las conclusiones del informe, porque todos los resultados de rendimiento están condicionados por el entorno simulado. Repetir los tres estudios sobre máquinas físicas independientes, con latencia y ancho de banda reales, permitiría verificar cuáles de las conclusiones son propiedades de los algoritmos y cuáles artefactos del entorno. La predicción concreta que quedaría a prueba es que la ventaja de All-Reduce sobre Parameter Server debería ampliarse, dado que la red rápida del entorno simulado atenúa el costo de comunicación que penaliza al servidor central. En la misma línea, un despliegue real permitiría por fin ejercitar el escenario en el que las épocas offline tienen sentido, que es aquel en el que el costo de comunicación domina.

Caracterizar Strategy-Switch en un régimen que ejercite la conmutación. El algoritmo está implementado y conmuta efectivamente cuando la curva de pérdida se aplanan; lo que ocurrió es que, en los benchmarks ejecutados, el umbral de estabilidad tomado del trabajo original no llegó a alcanzarse dentro del presupuesto de épocas evaluado, de modo que esas corridas particulares quedaron registradas como sin conmutación. Queda pendiente caracterizar el algoritmo sobre configuraciones que sí la ejerciten, y ese es el objeto del tercer estudio experimental de este informe. En la misma línea, corresponde revisar la topología de la fase inicial de All-Reduce, que hoy se ejecuta sobre la totalidad de los nodos, incluidos los que luego serán promovidos a servidores, para que la comparación contra All-Reduce y Parameter Server puros se realice sobre la misma cantidad de trabajadores.

Resolver la actualización de parámetros libre de bloqueos, o retirarla. El trilema documentado en las lecciones aprendidas admite dos salidas, cada una con su renuncia, y lo que corresponde es elegir una de forma explícita en lugar de dejar la variante en el limbo. Cualquiera de las dos exige antes verificar empíricamente si el enfoque rinde con gradientes densos, que es el régimen en el que el sistema opera.

Multithreading del motor de aprendizaje. La paralelización con hilos se descartó por las razones de la lección 2, pero esa conclusión está acotada a MNIST y a modelos pequeños. Con modelos y

batches considerablemente mayores el balance podría invertirse, y el trabajo dejó la medición hecha como línea de base contra la cual comparar.

Consumo de conjuntos de datos por flujo. El sistema carga las particiones del conjunto de datos en memoria. Un mecanismo de lectura por flujo desde disco permitiría trabajar con conjuntos que no entran en la memoria de un nodo, que es uno de los dos motivos originales por los que se distribuye un entrenamiento y el que este trabajo no llegó a ejercitar.

Ampliación del motor de redes neuronales. Quedan pendientes las convoluciones apiladas y una revisión de la función de pérdida de entropía cruzada para que valide que su entrada sea una distribución de probabilidad. Una optimización conocida es fusionar softmax con la entropía cruzada: hoy softmax es una capa ordinaria cuyo jacobiano se compone con el gradiente de la pérdida, lo cual es matemáticamente equivalente pero cuesta una pasada adicional por fila.

Deuda técnica registrada. Se dejan documentados los defectos conocidos al cierre, para que quien retome el trabajo sepa dónde pisar. La propagación hacia atrás del *max-pooling* pisa el gradiente cuando el paso es menor que el tamaño del filtro. La propagación hacia atrás de la ReLU con pendiente negativa es incorrecta, aunque el caso estándar es correcto y ningún modelo evaluado la utiliza. El script de descarga de MNIST crea un directorio donde debería crear un archivo. Y en cobertura de pruebas, las dos pruebas de integración existentes carecen de aserciones y funcionan como pruebas de humo, mientras que el anillo de All-Reduce, que es el algoritmo más intrincado del sistema, tiene su corrección verificada por sus resultados de convergencia y no por pruebas unitarias.

Evaluación aislada de las técnicas de reducción de tráfico. El sistema implementa codificación de gradientes en media precisión y un protocolo de gradientes dispersos con acumulación del residuo. Ambas se utilizaron en los experimentos, pero no se evaluaron de forma aislada: no se midió el compromiso entre la tasa de compresión y la degradación de la convergencia. Es un estudio autocontenido, de alcance acotado y con toda la instrumentación ya disponible, que sería el candidato natural para un cuarto estudio.

16 Conclusiones

El trabajo se propuso estudiar las principales estrategias de entrenamiento distribuido de modelos de aprendizaje profundo y construir un sistema que permitiera compararlas de forma controlada. Ese objetivo se cumplió.

Sobre el producto. Se construyó O.N.O., un sistema distribuido de entrenamiento que incluye una biblioteca de redes neuronales propia, una capa de comunicación con su propio protocolo, un plano de control y tres interfaces de uso. Sobre esa base se implementaron los tres algoritmos de referencia del área, se agregaron técnicas de reducción del tráfico de red y se instrumentó todo con una suite de experimentación reproducible.

Sobre los resultados. Los estudios experimentales muestran que, en un entorno homogéneo y sincrónico, All-Reduce y Parameter Server convergen a la misma exactitud, y que All-Reduce la alcanza en menos tiempo porque el servidor central introduce un punto de serialización; que la ventaja de All-Reduce se reduce al crecer el modelo, porque el fragmentado de parámetros reparte mejor la presión de comunicación; y que postergar la sincronización mediante épocas offline degrada la exactitud de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva. Los resultados de rendimiento están acotados al clúster simulado descrito en *Experimentación y validación*; los de convergencia y exactitud no dependen de esa limitación.

Sobre la contribución. El trabajo no innova en algoritmos: los tres implementados están publicados. La contribución es el instrumento, y responde exactamente al problema detectado en la propuesta: la ausencia de una base común sobre la cual comparar estrategias sin que la diferencia medida mezcle el efecto del algoritmo con el de su implementación. La evidencia de que esa base funciona es, paradójicamente, el resultado que menos llama la atención del informe: que las dos estrategias converjan a la misma exactitud.

Sobre la formación. El trabajo obligó a atravesar de punta a punta tres dominios que la carrera aborda por separado. El de los sistemas distribuidos, donde la lección más dura fue descubrir dónde termina exactamente el alcance de las garantías de un lenguaje seguro. El del aprendizaje profundo, donde derivar y depurar la propagación hacia atrás a mano, sin la red de contención de la diferenciación automática, dejó una comprensión del entrenamiento que ningún framework habría permitido adquirir. Y el de la ingeniería de software, donde la lección central fue que la instrumentación no es un anexo de medición sino una herramienta de detección, según se detalla en *Riesgos materializados y lecciones aprendidas*.

Se cierra el trabajo con un sistema funcionando, con evidencia experimental que lo respalda, con las limitaciones de esa evidencia declaradas y con las líneas que quedaron abiertas documentadas al detalle suficiente para que otro las retome.

17 Aportes individuales

El desarrollo del trabajo se planteó como una responsabilidad compartida: las etapas de análisis, revisión de código y pruebas fueron llevadas adelante por los tres integrantes en todas las tareas. Sin perjuicio de ello, cada integrante concentró su foco principal en un conjunto de componentes, que se detalla a continuación y que se corresponde con lo previsto en la propuesta.

Alejo Ordoñez (Padrón 108397). Núcleo de aprendizaje automático: el modelo, las capas, los optimizadores y las funciones de pérdida. Las capas convolucionales y de *max-pooling*, incluida la reformulación de la convolución como multiplicación de matrices que produjo la mejora final de rendimiento del motor. El manejo y la distribución de los conjuntos de datos. La monografía sobre *Strategy-Switch*, que constituye el tercer estudio experimental del informe.

Lorenzo Minervino (Padrón 107863). El módulo del Worker y la estrategia *All-Reduce* en anillo. La interfaz de funciones externas en Python y la interfaz interactiva de terminal. La suite de benchmarks y el análisis experimental de resultados. La monografía comparativa entre *Parameter Server* y *All-Reduce*, que constituye el primer estudio experimental del informe.

Marcos Bianchi Fernández (Padrón 108921). Los algoritmos de entrenamiento distribuido y su coordinación. El módulo de comunicaciones y el protocolo entre nodos, incluidas las técnicas de reducción de tráfico. El módulo del Servidor de parámetros. La monografía sobre el impacto de las épocas offline, que constituye el segundo estudio experimental del informe.

Cuadro 12: Distribución del trabajo individual por tipo de tarea.

	Alejo Ordoñez	Lorenzo Minervino	Marcos Bianchi Fernández
Cantidad de horas insumidas	<i>[a completar]</i>	<i>[a completar]</i>	<i>[a completar]</i>
Preparación de la propuesta	X	X	X
Relevamiento del estado del arte	X	X	X
Diseño de la arquitectura del sistema	X	X	X
Capa de comunicaciones y protocolo			X
Motor de redes neuronales	X		
Capas convolucionales y max-pooling	X		
Manejo y distribución de datasets	X		
Parameter Server			X

	Alejo Ordoñez	Lorenzo Minervino	Marcos Bianchi Fernández
Worker y All-Reduce en anillo		X	
Strategy-Switch		X	X
Orchestrator y coordinación		X	X
Optimizaciones de comunicación			X
Optimizaciones de sincronización		X	X
Optimización del motor de aprendizaje	X		
Interfaz de funciones externas en Python		X	
Interfaz interactiva de terminal		X	
Entornos de simulación con Docker		X	X
Definición de casos de prueba	X	X	X
Revisión de código	X	X	X
Suite de benchmarks		X	
Análisis experimental de resultados		X	
Monografía: All-Reduce frente a Parameter Server		X	
Monografía: impacto de las épocas offline			X
Monografía: Strategy-Switch	X		
Documentación técnica del sistema	X	X	X
Elaboración del Informe Final	X	X	X

18 Referencias

- Aji, A. F., & Heafield, K. (2017). Sparse Communication for Distributed Gradient Descent. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 440-445. <https://doi.org/10.18653/v1/D17-1045>
- Ben-Nun, T., & Hoeffler, T. (2019). Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis. *ACM Computing Surveys*, 52(4). <https://doi.org/10.1145/3320060>
- Dehghani, M., & Yazdanparast, Z. (2023). From Distributed Machine to Distributed Deep Learning: A Comprehensive Survey. *arXiv preprint arXiv:2307.05232*. Recuperado de <https://arxiv.org/abs/2307.05232>
- Glorot, X., & Bengio, Y. (2010). Understanding the Difficulty of Training Deep Feedforward Neural Networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, 249-256.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision*, 1026-1034.
- Jiang, Y., Zhu, Y., Lan, C., Yi, B., Cui, Y., & Guo, C. (2020). A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. *Proceedings of OSDI '20*.
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*. Recuperado de <https://arxiv.org/abs/1412.6980>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., ... Su, B.-Y. (2014). Scaling distributed machine learning with the parameter server. *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation*, 583-598. USA: USENIX Association.
- Li, Z., Davis, J., & Jarvis, S. (2017). An Efficient Task-based All-Reduce for Machine Learning Applications. *Proceedings of the Machine Learning on HPC Environments*. New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3146347.3146350>
- Lin, Y., Han, S., Mao, H., Wang, Y., & Dally, W. J. (2018). Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training. *International Conference on Learning Representations*. Recuperado de <https://arxiv.org/abs/1712.01887>
- McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. y. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *Artificial Intelligence and Statistics*, 1273-1282. PMLR.
- Niu, F., Recht, B., Ré, C., & Wright, S. J. (2011). Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. *Advances in Neural Information Processing Systems 24*. Recuperado de <https://arxiv.org/abs/1106.5730>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Recuperado de <https://arxiv.org/abs/1912.01703>
- Patarasuk, P., & Yuan, X. (2009). Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. *Journal of Parallel and Distributed Computing*, 69(2), 117-124. <https://doi.org/10.1016/j.jpdc.2008.09.002>
- Prechelt, L. (1998). Early Stopping — But When? En *Neural Networks: Tricks of the Trade* (pp. 55-69). Springer. https://doi.org/10.1007/3-540-49430-8_3

- Provatas, N., Chalas, I., Konstantinou, I., & Koziris, N. (2025). Strategy-Switch: From All-Reduce to Parameter Server for Faster Efficient Training. *IEEE Access, PP*, 1-1. <https://doi.org/10.1109/ACCESS.2025.3528248>
- Sergeev, A., & Del Balso, M. (2018). Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799*. Recuperado de <https://arxiv.org/abs/1802.05799>
- Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the Importance of Initialization and Momentum in Deep Learning. *Proceedings of the 30th International Conference on Machine Learning*, 1139-1147.
- TensorFlow Developers. (2021). *TensorFlow*. Zenodo. <https://doi.org/10.5281/zenodo.4758419>
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv preprint arXiv:1708.07747*. Recuperado de <https://arxiv.org/abs/1708.07747>

19 Anexos

19.1 Anexo A: Repositorio del proyecto

El código fuente del trabajo, junto con su documentación técnica y el historial completo de su desarrollo, se encuentra alojado en el siguiente repositorio público, que se mantendrá disponible al menos hasta un año después de la defensa:

<https://github.com/lminervino18/oxidized-neural-orchestra>

El repositorio está organizado como un *workspace* de Cargo, la herramienta de construcción de Rust, y se divide en los siguientes módulos:

- **machine_learning**: el núcleo de redes neuronales, que comprende las capas, los optimizadores, las funciones de pérdida y la representación de tensores.
- **comms**: el middleware de comunicación entre nodos.
- **parameter_server**: la implementación de la estrategia *Parameter Server*.
- **worker**: el runtime del nodo trabajador, que contiene además la implementación en anillo de *All-Reduce*.
- **orchestrator**: el plano de control, que asigna los roles de servidor y de trabajador y dirige la ejecución de un entrenamiento.
- **node**: el proceso de cómputo, agnóstico del rol que le sea asignado.
- **orchestra-py**: la biblioteca de Python, construida sobre la interfaz de funciones externas.
- **orchestui**: la interfaz interactiva de terminal.
- **benchmarks**: la suite de experimentación y los generadores de gráficos.
- **docker**: los scripts que levantan los entornos de simulación multinodo.
- **docs**: la propuesta, el presente informe y la documentación de diseño del sistema.

19.2 Anexo B: Documentación técnica

Dentro del repositorio se encuentran disponibles los siguientes documentos, referenciados a lo largo de este informe:

- **docs/config-schema.md**: la referencia canónica del esquema de configuración de un entrenamiento. Es la única fuente de verdad del formato aceptado por las tres interfaces del sistema.
- **docs/architecture-draft.md**: el borrador de arquitectura que sostuvo las discusiones de diseño de las etapas iniciales.
- **docs/report/roadmap.md**: la reconstrucción detallada del recorrido conceptual del proyecto a partir del historial completo del repositorio, con las decisiones de arquitectura, los debates que las motivaron y su fundamentación bibliográfica. Es el documento de apoyo del que se derivan las secciones de *Metodología aplicada*, *Cronograma* y *Riesgos materializados y lecciones aprendidas*.
- **benchmarks/README.md**: la documentación de la suite de experimentación, con las configuraciones de cada campaña y las instrucciones para reproducirlas.
- Los **README.md** de cada crate, con la documentación de sus abstracciones principales.

19.3 Anexo C: Reproducibilidad de los experimentos

Los estudios experimentales presentados en este informe son reproducibles a partir del material versionado en el repositorio. Para cada campaña se conservan las configuraciones que la generaron, los resultados crudos y los resultados procesados, junto con los scripts que producen las figuras y las tablas a partir de ellos.

El material del primer estudio se encuentra en `doc/monography_all_reduce_vs_ps/`, e incluye el arnés de experimentación autocontenido, los archivos de configuración de los cuatro experimentos, los resultados en formato crudo y procesado, y los generadores de figuras y tablas. La campaña completa de benchmarks del sistema se ejecuta mediante `benchmarks/run_issue_benchmarks.py`; su última corrida completa abarcó 38 configuraciones sin fallos, con un tiempo total de cómputo de 5 horas y 39 minutos, y con tres repeticiones en las suites de throughput y de escalabilidad para reportar media y desvío.

Las semillas de inicialización de los parámetros están fijadas en todas las configuraciones. Debe tenerse presente, no obstante, que las mediciones de tiempo dependen del hardware y de la carga de la máquina en la que se ejecuten, de modo que la reproducibilidad exacta alcanza a las métricas de convergencia y de exactitud, y no a las de rendimiento.

19.4 Anexo D: Monografías

Los dos primeros estudios experimentales de este informe fueron desarrollados originalmente como monografías individuales en el marco de la materia Aprendizaje Profundo, y se presentan aquí en forma condensada y reorientada al sistema. Los documentos completos, con su desarrollo extendido, se encuentran disponibles en el repositorio.